

REPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE

MINISTRE DE L'ENSEIGNEMENT
SUPERIEUR

INSTITUT DE HAUTE
TECHNOLOGIE
ET DE MANAGEMENT



REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND

MINISTRY OF HIGHER
EDUCATION

INSTITUTE OF HIGH
TECHNOLOGY
AND MANAGEMENT



RAPPORT DE STAGE

CONCEPTION ET MISE EN PLACE D'UN SECURITY OPERATIONS CENTER (SOC) OPEN-SOURCE : CAS DE PANESS

Mémoire de fin de cycle présenté en vue de l'obtention de la

LICENCE PROFESSIONNELLE

Option : Réseaux et Sécurité Informatique des systèmes informatiques

Rédige et présente par :
SAMNICK BIGA RAOUL AUBIN

Sous l'encadrement :

Académique:
M.TEKA
WILFRIED

Professional :
M.AWOUAFACK
FABIEN

a mis en forme : Interligne : Multiple 1,08 li

a mis en forme : Haut: (Personnaliser, Bleu, 30,8 pt Épaisseur du trait, Marge : 25 pt Espacement des bordures :), Bas: (Personnaliser, Bleu, 30,8 pt Épaisseur du trait, Marge : 25 pt Espacement des bordures :), Gauche: (Personnaliser, Bleu, 30,8 pt Épaisseur du trait, Marge : 25 pt Espacement des bordures :), Droite: (Personnaliser, Bleu, 30,8 pt Épaisseur du trait), Première page différente

a mis en forme : Interligne : Multiple 1,08 li

a mis en forme : Gauche, Interligne : Multiple 1,08 li

a mis en forme : Espace Après : 8 pt, Interligne : 1,5 ligne

a mis en forme : Gauche

a mis en forme : Interligne : Multiple 1,08 li

a mis en forme : Centré, Interligne : Multiple 1,08 li

a mis en forme : Interligne : Multiple 1,08 li

|

|

a mis en forme : Police :10 pt, Gras

DÉDICACE

Nous dédions ce présent rapport aux membres de nos différentes familles, qui ont toujours été là pour nous soutenir tout au long de nos études et qui nous ont toujours donné un magnifique modèle de labeur et de persévérance. Nous espérons qu'elles trouveront dans notre travail toute notre reconnaissance et tout notre amour. Que ce travail soit le témoignage de ma profonde affection et de ma gratitude.

REMERCIEMENTS

Je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué à la réalisation de ce mémoire.

Mes remerciements s'adressent en premier lieu au corps pédagogique et administratif de l'IHTM pour la qualité de la formation dispensée.

Je remercie particulièrement M. TEKA, mon encadreur académique, pour ses conseils avisés, sa disponibilité et la rigueur de son suivi tout au long de la rédaction.

Je remercie également M. FABIEN, mon encadreur professionnel au sein de PANESS, pour son accompagnement technique et la confiance qu'il m'a accordée.

Je remercie l'ensemble de l'équipe de PANESS pour leur accueil chaleureux et leur disponibilité, qui m'ont permis de travailler dans d'excellentes conditions.

Enfin, je remercie ma famille et mes amis pour leur soutien moral constant.

RÉSUMÉ

Face à la sophistication croissante des cybermenaces ciblant les organisations africaines, la mise en place d'un Security Operations Center (SOC) constitue une réponse stratégique indispensable. Ce mémoire présente la conception et le déploiement d'un SOC entièrement fonctionnel basé sur des outils open-source, réalisé dans le cadre d'un stage professionnel au sein de l'entreprise PANESS à Yaoundé, Cameroun.

La méthodologie adoptée repose sur l'intégration progressive d'une pile technologique complète : Wazuh comme SIEM/EDR central, Suricata pour la détection d'intrusion réseau (NIDS), Auditd et Sysmon pour la supervision des terminaux, YARA et VirusTotal pour l'analyse de malware enrichie par l'IA DeepSeek, MISP pour le renseignement sur les menaces (Threat Intelligence), DFIR-IRIS pour la gestion des incidents, Shuffle pour l'automatisation des réponses (SOAR), et un bot Telegram pour la notification en temps réel des analystes.

L'infrastructure a été déployée dans un environnement de laboratoire virtualisé sous VirtualBox et validée par des scénarios d'attaque simulée. Les résultats démontrent qu'une solution robuste, intégrée et économiquement accessible peut être construite exclusivement à partir de technologies open-source.

Mots-clés : SOC, SIEM, EDR, Wazuh, MISP, YARA, open-source, cybersécurité, Cameroun.

ABSTRACT

Faced with the growing sophistication of cyber threats targeting African organizations, establishing a Security Operations Center (SOC) represents an indispensable strategic response. This report presents the design and deployment of a fully functional SOC built exclusively with open-source tools, carried out during a professional internship at the company PANESS in Yaoundé, Cameroon.

The methodology relies on the progressive integration of a comprehensive technology stack : Wazuh as the central SIEM/EDR, Suricata for network intrusion detection, Auditd and Sysmon for endpoint monitoring, YARA and VirusTotal for AI-enriched malware analysis, MISP for threat intelligence, DFIR-IRIS for incident case management, Shuffle for response automation, and a Telegram bot for real-time analyst notification.

The infrastructure was deployed in a VirtualBox virtualized lab and validated through simulated attack scenarios. Results demonstrate that a robust, integrated and cost-effective security solution can be built entirely from open-source technologies.

Keywords: SOC, SIEM, EDR, Wazuh, MISP, YARA, open-source, cybersecurity, Cameroon.

SOMMAIRE

Dédicace	1
Remerciements	3
Résumé	4
Abstract.....	5
Sommaire	6
Liste des figures.....	7
Liste des tableaux.....	12
Liste des annexes.....	13
Liste des abréviations	14
Introduction générale	15
Chapitre 1 — Présentation de l'entreprise d'accueil : PANESS	16
Chapitre 2 — Cadre conceptuel	18
Chapitre 3 — Problématique, objectifs et architecture retenue.....	20
Chapitre 4 — Mise en œuvre de l'infrastructure SOC.....	23
Chapitre 5 — Validation et résultats	56
Conclusion générale et perspectives	60
Références bibliographiques.....	61
Table des matières.....	62

LISTE DES FIGURES

1. Figure	1
<i>Architecture globale du SOC open-source.</i>	
2. Figure	2
<i>Réseau NAT « SOC-Network » dans VirtualBox (192.168.100.0/24).</i>	
3. Figure	3
<i>VM wazuh-server (OVA) importée et configurée dans VirtualBox.</i>	
4. Figure	4
<i>Configuration IP statique via nmcli sur wazuh-server (Amazon Linux 2023).</i>	
5. Figure	5
<i>Vérification des 5 machines virtuelles dans VirtualBox.</i>	
6. Figure	6
<i>Connectivité réseau confirmée par pings croisés.</i>	
7. Figure	7
<i>Services Wazuh actifs sur la VM wazuh-server (systemctl status).</i>	
8. Figure	8
<i>Accès au Wazuh Dashboard via https://192.168.20.45.</i>	
9. Figure	9
<i>Authentification Dashboard Wazuh (admin/admin par défaut).</i>	
10. Figure	10
<i>Tableau de bord principal de Wazuh après connexion.</i>	
11. Figure	11
<i>Installation agent Wazuh sur agent-linux via apt.</i>	
12. Figure	12
<i>Service wazuh-agent actif sur agent-linux (systemctl status).</i>	

13. Figure	13
<i>Installation agent Wazuh sur agent-windows via PowerShell.</i>	
14. Figure	14
<i>Dashboard Wazuh affichant 3 agents Active.</i>	
15. Figure	15
<i>Liste CDB créée : /var/ossec/etc/lists/suspicious-programs.</i>	
16. Figure	16
<i>Règles FIM, Auditd et Sysmon dans local_rules.xml.</i>	
17. Figure	17
<i>Événement FIM détecté dans le Dashboard Wazuh.</i>	
18. Figure	18
<i>Alerte Auditd de commande suspecte (niveau 12).</i>	
19. Figure	19
<i>Alerte Auditd de commande suspecte (niveau 12).</i>	
20. Figure	20
<i>Installation Suricata sur wazuh-server (dnf install).</i>	
21. Figure	21
<i>Configuration suricata.yaml avec HOME_NET et interface réseau.</i>	
22. Figure	22
<i>Téléchargement des règles Emerging Threats.</i>	
23. Figure	23
<i>Suricata actif et générant des alertes (systemctl status).</i>	
24. Figure	24
<i>Alertes Suricata visibles dans le Dashboard Wazuh.</i>	
25. Figure	25
<i>Scan nmap depuis kali-attacker générant alertes Suricata.</i>	

26. Figure	26
<i>Compilation YARA v4.5.1 sur agent-linux (./configure && make).</i>	
27. Figure	27
<i>Vérification YARA : yara --version affiche la version compilée.</i>	
28. Figure	28
<i>Téléchargement des règles Valhalla pour agent-linux.</i>	
29. Figure	29
<i>Script yara.sh créée dans /var/ossec/active-response/bin/.</i>	
30. Figure	30
<i>Test YARA manual sur un fichier EICAR.</i>	
31. Figure	31
<i>Installation binaire YARA (yara64.exe) sur agent-windows.</i>	
32. Figure	32
<i>Script yara.py créé pour agent-windows.</i>	
33. Figure	33
<i>Compilation yara.py en yara.exe via PyInstaller.</i>	
34. Figure	34
<i>Test YARA manual sur Windows (yara64.exe sur fichier test).</i>	
35. Figure	35
<i>Décodeurs YARA dans /var/ossec/etc/decoders/local_decoder.xml.</i>	
36. Figure	36
<i>Règles YARA (108000-108003) dans local_rules.xml.</i>	
37. Figure	37
<i>Script yara.sh créé sur wazuh-server avec permissions root:wazuh.</i>	
38. Figure	38
<i>Script remove-threat.sh pour suppression automatique malware.</i>	

39. Figure	39
<i>Intégration VirusTotal configurée dans ossec.conf.</i>	
40. Figure	40
<i>Alerte 108001 affichant détection YARA positive.</i>	
41. Figure	41
<i>Alerte 108002 confirmant suppression automatique du fichier.</i>	
42. Figure	42
<i>Enrichissement DeepSeek visible dans le champ YARA.chatgpt_response.</i>	
43. Figure	43
<i>Déploiement MISP via Docker sur soc-services.</i>	
44. Figure	44
<i>Interface web MISP accessible via https://192.168.100.20:1443.</i>	
45. Figure	45
<i>Script custom-misp.py créé sur wazuh-server.</i>	
46. Figure	46
<i>Intégration MISP dans ossec.conf.</i>	
47. Figure	47
<i>Alerte 100622 signalant détection IOC malveillant via MISP.</i>	
48. Figure	48
<i>Déploiement IRIS via Docker sur soc-services.</i>	
49. Figure	49
<i>Interface IRIS accessible via https://192.168.100.20:8443.</i>	
50. Figure	50
<i>Script custom-wazuh_iris.py créé sur wazuh-server.</i>	
51. Figure	51
<i>Intégration IRIS dans ossec.conf (webhook).</i>	

52. Figure	52
<i>Cas d'incident créé automatiquement dans IRIS.</i>	
53. Figure	53
<i>Déploiement Shuffle via Docker sur soc-services.</i>	
54. Figure	54
<i>Workflow Shuffle configuré pour extraction IOC et VirusTotal.</i>	
55. Figure	55
<i>Webhook Shuffle intégré dans ossec.conf.</i>	
56. Figure	56
<i>Script custom-whatsapp.py créé sur wazuh-server.</i>	
57. Figure	57
<i>Message du bot Telegram reçu suite à alerte critique de Wazuh.</i>	
58. Figure	58
<i>Notification Shuffle pour vérification VirusTotal sur Telegram.</i>	
59. Figure	59
<i>Chronologie des alertes Wazuh : FIM → YARA → suppression automatique.</i>	
60. Figure	60
<i>Alerte Suricata/Wazuh générée suite au scan nmap.</i>	
61. Figure	61
<i>Alerte niveau 12 confirmant la détection de « nc » via la liste CDB.</i>	
62. Figure	62
<i>Détection de force brute SSH avec corrélation MISP de l'IP source.</i>	
63. Figure	63
<i>Vue d'ensemble des alertes dans le Dashboard Wazuh lors des tests.</i>	
64. Figure	64
<i>Notifications Telegram reçues correspondant aux quatre scénarios.</i>	

LISTE DES TABLEAUX

65. Tableau	1
<i>Outils retenus et rôle dans le SOC.</i>	
66. Tableau	2
<i>Machines virtuelles, systèmes et adresses IP.</i>	

LISTE DES ANNEXES

- 67. Annexe 1. Fichier de configuration ossec.conf (extraits principaux)
- 68. Annexe 2. Scripts d'intégration personnalisés (MISP, IRIS, YARA)
- 69. Annexe 3. Règles de détection personnalisées (local_rules.xml)

LISTE DES ABRÉVIATIONS

SIGLE	SIGNIFICATION
API (Application Programming Interface)	Interface de Programmation d'Application (Application Programming Interface)
AWS	Amazon Web Services
CDB (Custom Database)	Base de données personnalisée (Custom Database)
DFIR (Digital Forensics and Incident Response)	Informatique légale et réponse aux incidents (Digital Forensics and Incident Response)
EDR	Détection et réponse aux menaces sur les terminaux (Endpoint Detection and Response)
FIM (File Integrity Monitoring)	Surveillance de l'intégrité des fichiers (File Integrity Monitoring)
HIDS	Système de détection d'intrusion basé sur l'hôte (Host-based Intrusion Detection System)
IHTM (Institut de Haute Technologie et de Management)	Institut de Haute Technologie et de Management
IOC	Indicateur de compromission (Indicator of Compromise)
IRIS	Plateforme DFIR-IRIS (Digital Forensics Incident Response Platform)
ISAC	Centre de partage et d'analyse de l'information (Information Sharing and Analysis Center)
MISP (Malware Information Sharing Platform)	Plateforme de partage d'informations sur les malwares (Malware Information Sharing Platform)
NAT (Network Address Translation)	Traduction d'adresses réseau (Network Address Translation)
NIDS	Système de détection d'intrusion réseau (Network-based Intrusion Detection System)
OSINT	Renseignement d'origine source ouverte (Open Source Intelligence)
OVA (Open Virtual Appliance)	Archive de virtualisation ouverte (Open Virtual Appliance)
PME	Petites et Moyennes Entreprises
SIEM	Gestion des informations et des événements de sécurité (Security Information and Event Management)
SOAR	Orchestration, automatisation et réponse aux incidents de sécurité (Security Orchestration, Automation and Response)
SOC	Centre d'opérations de sécurité (Security Operations Center)
SSH (Secure Shell)	Secure Shell (Protocole de communication sécurisé)
VM	Machine Virtuelle (Virtual Machine)
YARA (Yet Another Recursive Acronym)	Yet Another Recursive Acronym (Outil de détection de malwares)

Introduction générale

Les réseaux d'entreprise modernes constituent des cibles privilégiées pour les cyberattaques, en raison de leur complexité et de la valeur des données qu'ils hébergent. Qu'il s'agisse de rançongiciels, de vols de données ou d'intrusions ciblées, les menaces évoluent en permanence, rendant les défenses périmétriques traditionnelles insuffisantes (IBM Security, 2024). Dans ce contexte, la mise en place d'un Security Operations Center (SOC) s'impose comme une nécessité pour toute organisation soucieuse de protéger son patrimoine numérique.

Ce mémoire présente la conception et le déploiement d'un SOC fonctionnel basé exclusivement sur des outils open-source, réalisé dans le cadre d'un stage professionnel au sein de PANESS à Yaoundé. L'objectif est de démontrer qu'une solution de sécurité robuste et complète peut être construite sans recourir aux licences commerciales coûteuses de solutions telles que Splunk ou IBM QRadar.

Ce document est structuré en cinq chapitres. Le premier présente l'entreprise d'accueil. Le second pose les bases conceptuelles. Le troisième expose la problématique, les objectifs et l'architecture retenue. Le quatrième détaille la mise en œuvre technique, phase par phase. Le cinquième présente les scénarios de validation et les résultats obtenus.

Chapitre 1 : Présentation de l'Entreprise d'Accueil : PANESS

1.1 Historique et Présentation Générale

PANESS est un cabinet de conseil, d'informatique et de formation fondé en 1994 et basé à Douala, au Cameroun. Acteur de premier plan en Afrique centrale et de l'Ouest, le groupe accompagne les organisations dans leur développement organisationnel, managérial et technologique au travers de trois filiales majeures.

1.2 Les Filiales du Groupe

1.2.1 PANESS Conseil

Filiale historique du groupe, PANESS Conseil est spécialisée en ingénierie des ressources humaines, management, stratégie d'entreprise et démarche qualité. Elle est également à l'origine des instituts supérieurs PIST Université et IHTM Université.

1.2.2 PANESS Digital Center

PANESS Digital Center se consacre à la création et à la refonte de sites web, blogs et plateformes e-commerce, en proposant des outils digitaux adaptés au contexte africain.

1.2.3 PANESS IT

PANESS IT est le centre des services informatiques du groupe. Il intervient dans le conseil, l'ingénierie applicative, l'infrastructure et la formation technique. C'est au sein de cette filiale que s'est déroulé le stage ayant donné lieu à ce mémoire.

1.3 Cadre du Stage

1.3.1 Mission Confiée

Le stage s'est déroulé au sein du pôle technique de PANESS IT, sous la supervision de M. TEKA. La mission consistait à concevoir, déployer et documenter un SOC complet basé sur des outils open-source dans un environnement de laboratoire virtualisé.

1.3.2 Tâches Réalisées

- Conception de l'architecture réseau et déploiement de l'environnement VirtualBox.
- Installation et configuration de la plateforme Wazuh (SIEM/EDR).
- Déploiement des agents de supervision sur des postes Linux et Windows.
- Mise en œuvre de Suricata (NIDS), YARA, VirusTotal et DeepSeek AI.
- Intégration de MISP (Threat Intelligence), DFIR-IRIS (Case Management) et Shuffle (SOAR).
- Configuration d'un bot Telegram pour la notification en temps réel.
- Réalisation de scénarios d'attaque et documentation du projet.

1.3.3 Apports du Stage

Ce stage a permis de consolider des compétences techniques en administration système Linux et Windows, virtualisation, sécurité réseau et scripting d'intégration, tout en développant une rigueur méthodologique dans la conduite d'un projet informatique de bout en bout.

Chapitre 2 : Cadre Conceptuel

2.1 Définition des Concepts Clés

2.1.1 Security Operations Center (SOC)

Un SOC est une équipe et une infrastructure dédiées à la surveillance continue, la détection, l'analyse et la réponse aux incidents de sécurité informatique. Il constitue le cœur opérationnel de la stratégie de cyberdéfense d'une organisation (Bhatt et al., 2014).

2.1.2 SIEM et EDR

Un SIEM (Security Information and Event Management) collecte, corrèle et analyse en temps réel les journaux de sécurité issus de multiples sources. Une EDR (Endpoint Detection and Response) surveille les terminaux pour y détecter et répondre aux menaces avancées. Wazuh combine nativement ces deux fonctions (Wazuh, 2024).

2.1.3 NIDS et HIDS

Un NIDS (Network-based Intrusion Detection System) analyse le trafic réseau pour identifier des activités suspectes, tandis qu'un HIDS (Host-based Intrusion Detection System) surveille l'activité directement sur une machine. Suricata, Auditd et Sysmon illustrent ces approches dans ce projet.

2.1.4 Threat Intelligence et IOC

La Threat Intelligence désigne l'ensemble des connaissances sur les menaces actives, partagées pour renforcer la détection collective. Un IOC (Indicator of Compromise) est une donnée concrète — adresse IP, hachage de fichier, nom de domaine — signalant une activité malveillante probable (Hutchins et al., 2011).

2.1.5 SOAR et Case Management

Le SOAR (Security Orchestration, Automation and Response) regroupe les technologies permettant d'automatiser les réponses aux incidents. Le Case Management, assuré par des outils comme DFIR-IRIS, désigne la gestion structurée des incidents depuis la détection jusqu'à la résolution.

2.2 Nécessité d'un SOC dans le Contexte Actuel

Le paysage de la cybersécurité est marqué par une augmentation constante de la sophistication et de la fréquence des attaques. Selon l'IBM Security X-Force Threat Intelligence Index (2024), les organisations africaines sont de plus en plus ciblées. Pour les structures à ressources limitées, les solutions commerciales comme Splunk ou IBM QRadar, dont les licences dépassent souvent plusieurs dizaines de milliers de dollars annuels, sont inaccessibles. Les solutions open-source constituent une alternative sérieuse, Wazuh étant notamment déployé dans plus de 30 000 environnements actifs dans le monde (Wazuh, 2024).

Chapitre 3 : Problématique, Objectifs et Architecture Retenue

3.1 Problématique

La mise en place d'un SOC performant représente un investissement souvent hors de portée des PME. La problématique centrale de ce projet est : dans quelle mesure une solution SOC open-source, déployée rapidement via une image préconfigurée (OVA), peut-elle assurer une détection et une réponse aux incidents de sécurité comparables aux solutions commerciales ?

3.2 Objectifs du Projet

3.2.1 Objectif Général

Concevoir, déployer et valider un prototype de SOC open-source, centré sur Wazuh v4.14.5 (via OVA), capable d'assurer la détection, l'analyse et la réponse aux incidents dans un environnement simulé représentatif d'une infrastructure d'entreprise.

3.2.2 Objectifs Spécifiques

70. Déployer une infrastructure de virtualisation représentant un réseau d'entreprise.
71. Déployer la plateforme Wazuh v4.14.5 (OVA, Amazon Linux 2023).
72. Installer et configurer Suricata (NIDS) sur la VM wazuh-server.
73. Déployer les agents de supervision sur des terminaux Linux et Windows.
74. Mettre en œuvre la surveillance des terminaux via FIM, Auditd et Sysmon.
75. Mettre en place l'analyse de malware via YARA (agent-linux et agent-windows), VirusTotal et DeepSeek.
76. Intégrer MISP pour le renseignement sur les menaces.
77. Déployer DFIR-IRIS pour la gestion des incidents.

- 78. Automatiser les réponses avec Shuffle et notifier via Telegram.
- 79. Valider l'architecture par des scénarios d'attaque simulée.

3.3 Choix des Outils et Justification

Le choix des outils a été guidé par leur maturité, la richesse fonctionnelle, la vitalité de leur communauté et leur capacité d'intégration mutuelle. L'utilisation de l'OVA Wazuh v4.14.5 (Amazon Linux 2023) a été retenue pour sa stabilité et son déploiement rapide, évitant les complications liées aux installations manuelles. Suricata a été installé directement sur la VM wazuh-server pour centraliser la détection réseau. YARA a été déployé de manière distribuée : sur agent-linux pour la surveillance des fichiers Linux, et sur agent-windows pour celle des fichiers Windows. Le tableau 2 synthétise les briques retenues.

COMPOSANT	OUTIL	PLATEFORME / LOCALISATION
SIEM / EDR	Wazuh v4.14.5	wazuh-server (OVA, Amazon Linux 2023)
NIDS	Suricata	wazuh-server (installation directe)
Supervision Linux	Auditd	agent-linux (Debian 12)
Supervision Windows	Sysmon	agent-windows (Windows 10)
Analyse malware Linux	YARA	agent-linux (compilation source)
Analyse malware Windows	YARA	agent-windows (binaire)
Réputation fichiers	VirusTotal	intégration wazuh-server
Intelligence artificielle	DeepSeek	intégration wazuh-server
Threat Intelligence	MISP	soc-services (Docker)
Case Management	DFIR-IRIS	soc-services (Docker)
SOAR	Shuffle	soc-services (Docker)
Notification	Telegram (TelegramAPI)	intégration wazuh-server

Tableau 1
Outils retenus et localisation dans l'infrastructure.

3.4 Architecture Globale Retenue

L'architecture suit le cycle de vie complet d'une alerte : collecte depuis les terminaux et le réseau, centralisation et analyse par Wazuh, enrichissement par les modules d'analyse et de threat intelligence, puis transmission aux plateformes de réponse et de notification. La VM wazuh-server (OVA) constitue le centre du déploiement, hébergeant le SIEM/EDR (Wazuh), la détection réseau (Suricata) et les intégrations MISP/IRIS via webhooks. Les agents Linux et Windows remontent les données de supervision et de malware au serveur. La figure 1 illustre cette architecture.

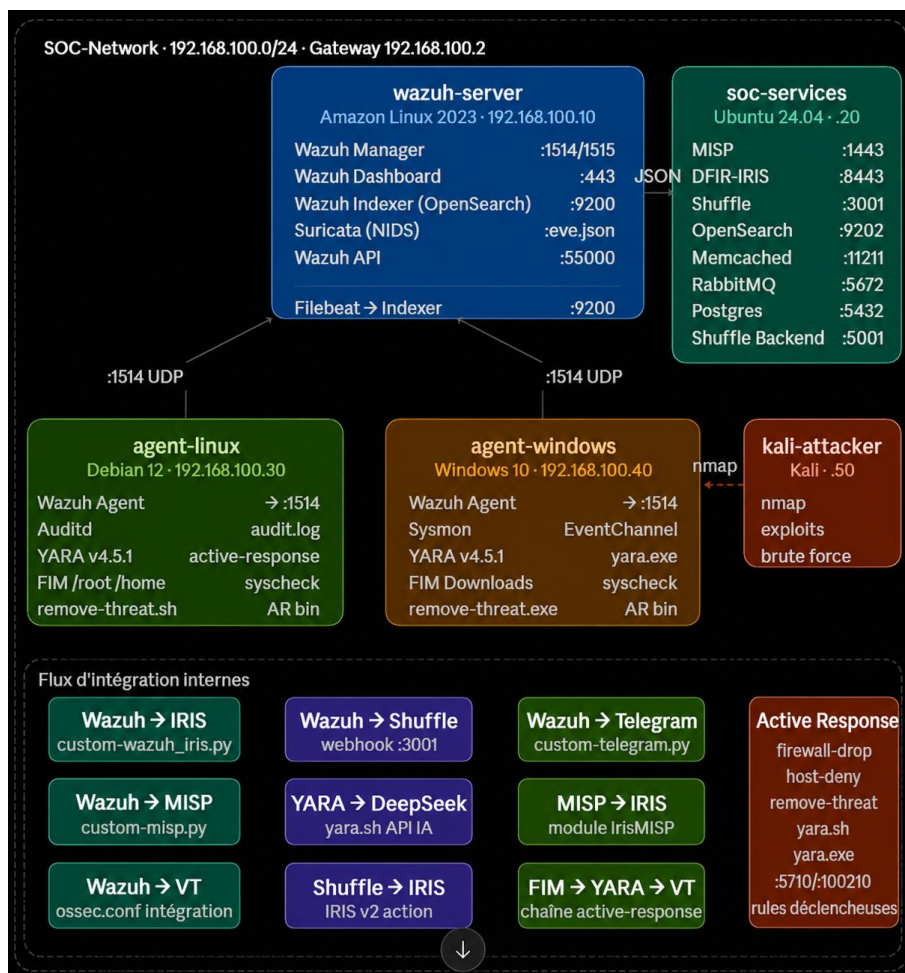


Figure 1. Architecture globale du SOC open-source.

Chapitre 4 : Mise en Œuvre de l'Infrastructure SOC

Ce chapitre décrit la mise en œuvre technique complète du SOC, organisée en neuf phases successives. L'utilisation de l'OVA Wazuh v4.14.5 a simplifié le déploiement initial. Suricata a été installé directement sur la VM wazuh-server pour éviter les complications liées aux installations distribuées. YARA a été installé de manière distribuée selon les besoins. Les emplacements des captures d'écran à insérer sont indiqués par des encadrés numérotés.

4.1 Phase 1 : Infrastructure de Virtualisation et Réseau

L'environnement de laboratoire a été construit sous VirtualBox. Un réseau NAT nommé SOC-Network (192.168.100.0/24) a été créé. Cinq machines virtuelles composent l'infrastructure, avec la VM wazuh-server utilisant l'OVA Wazuh v4.14.5 basée sur Amazon Linux 2023. Les autres machines hébergent les agents de supervision et la machine d'attaque.

VM	SYSTÈME	IP	RÔLE
wazuh-server	Amazon Linux 2023 (OVA)	192.168.100.10	SIEM/EDR + Suricata
soc-services	Ubuntu 24.04	192.168.100.20	MISP, IRIS, Shuffle
agent-linux	Debian 12	192.168.100.30	Agent + YARA
agent-windows	Windows 10	192.168.100.40	Agent + YARA
kali-attacker	Kali Linux	192.168.100.50	Machine d'attaque

Tableau 2
Machines virtuelles, systèmes et adresses IP.

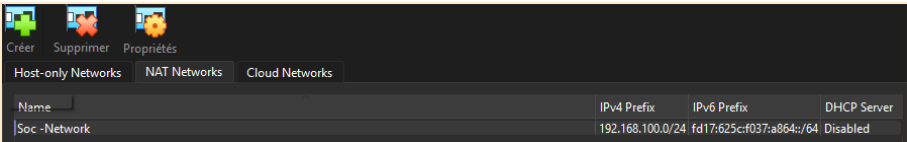


Figure 2
Réseau NAT « SOC-Network » dans VirtualBox (192.168.100.0/24).

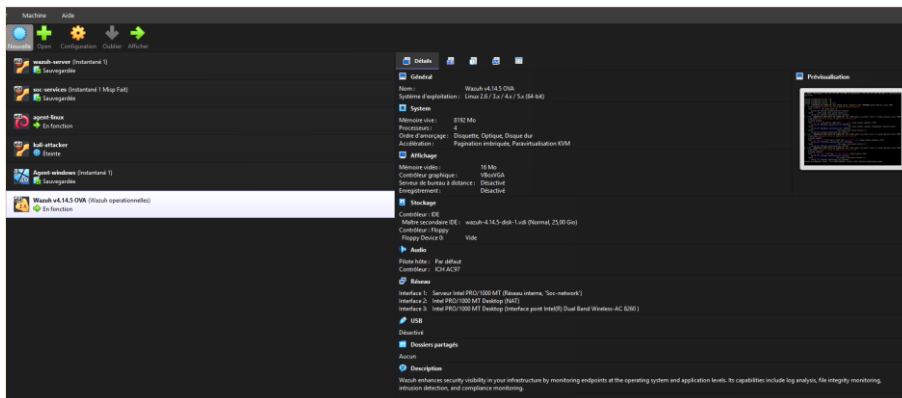


Figure 3
VM wazuh-server (OVA) importée et configurée dans VirtualBox.

```
[wazuh-user@wazuh-server ~]$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:59:8a:9c brd ff:ff:ff:ff:ff:ff
    altnam e np0s8
    inet 10.0.3.15/24 metric 1024 brd 10.0.3.255 scope global dynamic eth0
        valid_lft 73579sec preferred_lft 73579sec
    inet6 fd17:625c:f037:3:a00:27ff:fe59:8a9c/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86241sec preferred_lft 14241sec
    inet6 fe80::a00:27ff:fe59:8a9c/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c9:4f:2c brd ff:ff:ff:ff:ff:ff
    altnam e np0s9
    inet 192.168.20.45/24 metric 1024 brd 192.168.20.255 scope global dynamic eth1
        valid_lft 159980sec preferred_lft 159980sec
    inet6 fe80::a00:27ff:fec9:4f2c/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
4: eth2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ed:06:62 brd ff:ff:ff:ff:ff:ff
    altnam e np0s17
    inet 192.168.100.10/24 brd 192.168.100.255 scope global eth2
        valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feed:662/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
[wazuh-user@wazuh-server ~]$
```

Figure 4
Configuration IP statique via nmcli sur wazuh-server (Amazon Linux 2023).

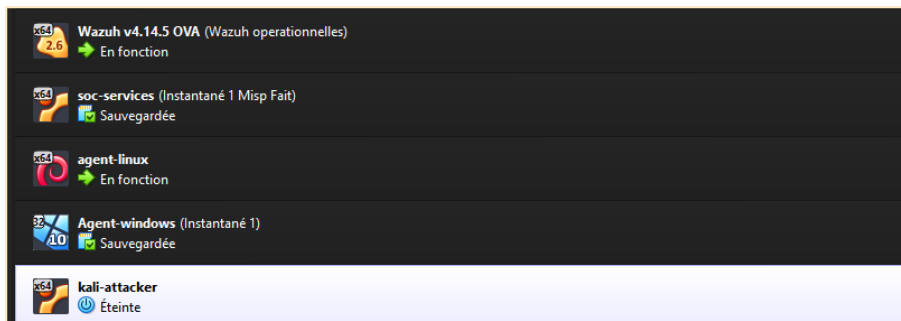


Figure 5

Vérification des 5 machines virtuelles dans VirtualBox.

```
[wazuh-user@wazuh-server ~]$ ping -c 4 192.168.100.30
PING 192.168.100.30 (192.168.100.30) 56(84) bytes of data.
64 bytes from 192.168.100.30: icmp_seq=1 ttl=64 time=7.46 ms
64 bytes from 192.168.100.30: icmp_seq=2 ttl=64 time=1.66 ms
64 bytes from 192.168.100.30: icmp_seq=3 ttl=64 time=3.08 ms
64 bytes from 192.168.100.30: icmp_seq=4 ttl=64 time=2.57 ms

--- 192.168.100.30 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.656/3.690/7.455/2.232 ms
[wazuh-user@wazuh-server ~]$ ping -c 4 192.168.100.20
PING 192.168.100.20 (192.168.100.20) 56(84) bytes of data.
64 bytes from 192.168.100.20: icmp_seq=1 ttl=64 time=5.54 ms
64 bytes from 192.168.100.20: icmp_seq=2 ttl=64 time=3.77 ms
64 bytes from 192.168.100.20: icmp_seq=3 ttl=64 time=2.43 ms
64 bytes from 192.168.100.20: icmp_seq=4 ttl=64 time=4.29 ms

--- 192.168.100.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 2.431/4.006/5.538/1.113 ms
[wazuh-user@wazuh-server ~]$
```

```

raoulbiga@agent-linux:~$ ping -c 4 192.168.100.10
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data.
64 bytes from 192.168.100.10: icmp_seq=1 ttl=127 time=4.12 ms
64 bytes from 192.168.100.10: icmp_seq=2 ttl=127 time=1.79 ms
64 bytes from 192.168.100.10: icmp_seq=3 ttl=127 time=3.94 ms
64 bytes from 192.168.100.10: icmp_seq=4 ttl=127 time=1.65 ms

--- 192.168.100.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.650/2.873/4.116/1.157 ms
raoulbiga@agent-linux:~$ ping -c 4 192.168.100.20
PING 192.168.100.20 (192.168.100.20) 56(84) bytes of data.
64 bytes from 192.168.100.20: icmp_seq=1 ttl=64 time=9.91 ms
64 bytes from 192.168.100.20: icmp_seq=2 ttl=64 time=2.01 ms
64 bytes from 192.168.100.20: icmp_seq=3 ttl=64 time=7.11 ms
64 bytes from 192.168.100.20: icmp_seq=4 ttl=64 time=4.18 ms

--- 192.168.100.20 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 2.010/5.803/9.914/2.983 ms
raoulbiga@agent-linux:~$

```

Figure 6

Connectivité réseau confirmée par pings croisés.

4.2 Phase 2 : Déploiement de Wazuh

L'OVA Wazuh v4.14.5 (Amazon Linux 2023) a été importée directement dans VirtualBox. Wazuh est préinstallé et préconfiguré sur cette image. L'étape de configuration consiste à vérifier que tous les services sont actifs (Wazuh Indexer, Wazuh Manager, Wazuh Dashboard, Filebeat) et à fixer la configuration API si nécessaire pour accéder au dashboard web.

```

[wazuh-user@wazuh-server ~]$ sudo systemctl is-active wazuh-manager wazuh-indexer wazuh-dashboard filebeat
active
active
active
active
[wazuh-user@wazuh-server ~]$

```

Figure 7

Services Wazuh actifs sur la VM wazuh-server (systemctl status).

 **Non sécurisé** <https://192.168.20.45/app/login?nextUrl=%2F>

Figure 8

Accès au Wazuh Dashboard via <https://192.168.20.45>.

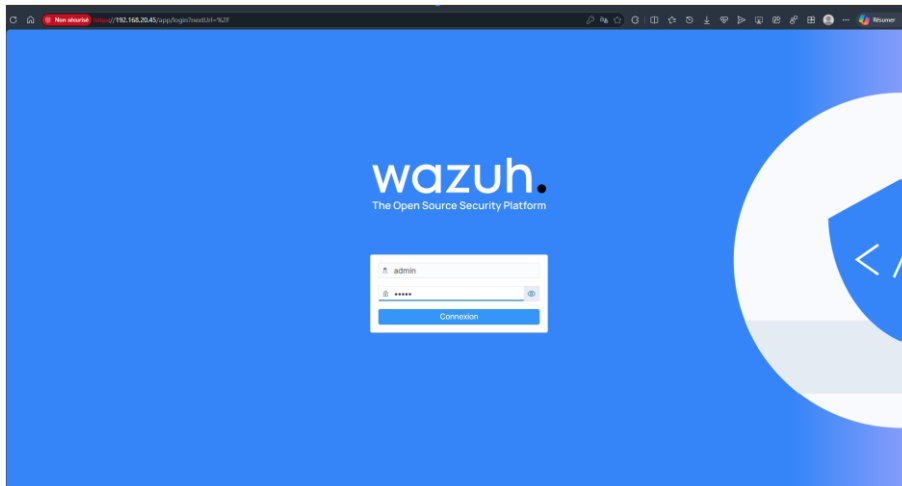


Figure 9

Authentification Dashboard Wazuh (admin/admin par défaut).

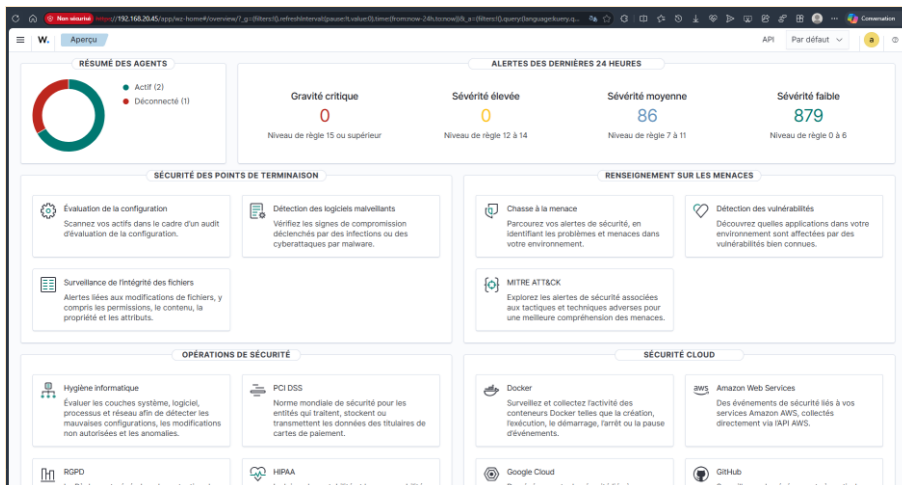


Figure 10

Tableau de bord principal de Wazuh après connexion.

4.3 Phase 3 : Agents Wazuh

Les agents Wazuh ont été déployés sur agent-linux (Debian 12) et agent-windows (Windows 10). Sur Debian, l'installation s'est faite via le gestionnaire de paquets après ajout

du dépôt Wazuh. Sur Windows, l'agent a été installé via un exécutable MSI en PowerShell administrateur. Chaque agent transmet automatiquement ses journaux au serveur central (192.168.100.10).

```
raoulbiga@agent-linux:~$ sudo apt install -y wazuh-agent
[sudo] Mot de passe de raoulbiga :
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets suivants seront mis à jour :
  wazuh-agent
1 mis à jour, 0 nouvellement installés, 0 à enlever et 36 non mis à jour.
Il est nécessaire de prendre 13,2 Mo dans les archives.
Après cette opération, 4 919 ko d'espace disque supplémentaires seront utilisés.
Réception de :1 https://packages.wazuh.com/4.x/apt stable/main amd64 wazuh-agent amd64 4.14.5-1 [13,2 MB]
13,2 Mo réceptionnés en 20s (650 ko/s)
Préconfiguration des paquets...
(Lecture de la base de données... 45147 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../wazuh-agent_4.14.5-1_amd64.deb ...
Dépaquetage de wazuh-agent (4.14.5-1) sur (4.12.0-1) ...
Paramétrage de wazuh-agent (4.14.5-1) ...

Fichier de configuration « /etc/systemd/system/wazuh-agent.service »
==> Supprimé (par vous ou par un script) depuis l'installation.
==> Le distributeur du paquet a fourni une version mise à jour.
Que voulez-vous faire ? Vos options sont les suivantes :
  Y ou I : installer la version du responsable du paquet
  N ou O : garder votre version actuellement installée
  D      : afficher les différences entre les versions
  Z      : suspendre ce processus pour examiner la situation
L'action par défaut garde votre version actuelle.
*** wazuh-agent.service (Y/I/N/O/D/Z) [défaut=N] ? n
raoulbiga@agent-linux:~$
```

Figure 11
Installation agent Wazuh sur agent-linux via apt.

```
raoulbiga@agent-linux:~$ sudo systemctl status wazuh-agent --no-pager | head -10
[sudo] Mot de passe de raoulbiga :
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-06-25 14:35:01 CEST; 1min 55s ago
   Process: 3865 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
   Tasks: 32 (limit: 2298)
   Memory: 66.5M
   CPU: 1min 9.228s
   CGroup: /system.slice/wazuh-agent.service
           └─3888 /var/ossec/bin/wazuh-execd
           └─3899 /var/ossec/bin/wazuh-agentd
raoulbiga@agent-linux:~$
```

Figure 12
Service wazuh-agent actif sur agent-linux (systemctl status).

```

Répertoire : C:\Program Files\ossec-agent

Mode                LastWriteTime         Length Name
----                -
d-----         6/4/2026   2:17 PM                active-response
d-----         6/4/2026   2:17 PM                bookmarks
d-----         6/4/2026   2:17 PM                incoming
d-----         6/6/2026   7:52 PM                logs
d-----         6/4/2026   2:17 PM                queue
d-----        6/24/2026  12:26 PM                rids
d-----         6/4/2026   2:17 PM                ruleset
d-----         6/4/2026   2:27 PM                shared
d-----         6/4/2026   2:17 PM                syscheck
d-----         6/6/2026   7:59 PM                tmp
d-----         6/4/2026   2:17 PM                upgrade
d-----         6/4/2026   2:17 PM                wodles
-a-----        6/25/2026   1:52 PM             43 .agent_info
-a-----        4/16/2026  12:55 PM          358704 agent-auth.exe
-a-----        4/16/2026  12:45 PM           2532 agent-auth.exe.manifest
-a-----        6/24/2026  12:25 PM             88 client.keys
-a-----        4/16/2026  12:56 PM          426784 dbsync.dll
-a-----        4/16/2026  12:50 PM           1277 help.txt
-a-----        4/16/2026  12:50 PM          15455 internal_options.conf
-a-----         6/4/2026   2:17 PM           9983 last-ossec.conf
-a-----        4/16/2026  12:56 PM          473904 libfimdb.dll
-a-----        4/16/2026  12:56 PM          147928 libgcc_s_dw2-1.dll
-a-----        4/16/2026  12:56 PM         2358072 libstdc++-6.dll
-a-----        4/16/2026  12:56 PM         8734000 libwazuhext.dll
-a-----        4/16/2026  12:56 PM          279856 libwazuhshared.dll
-a-----        4/16/2026  12:56 PM           60208 libwinpthread-1.dll
-a-----        4/16/2026  12:50 PM           25209 LICENSE.txt
-a-----        4/16/2026  12:50 PM           383 local_internal_options.conf
-a-----        4/16/2026  12:55 PM          361776 manage_agents.exe
-a-----        6/24/2026  12:45 PM          10519 ossec.conf
-a-----        6/25/2026   2:01 PM          16315 ossec.log
-a-----        4/16/2026  12:45 PM            51 profile-10.template
-a-----        4/16/2026  12:56 PM          336672 rsync.dll
-a-----        4/16/2026  12:56 PM          508208 syscollector.dll
-a-----        4/16/2026  12:56 PM          942880 sysinfo.dll
-a-----        4/16/2026  12:50 PM            78 VERSION.json
-a-----        4/16/2026  12:45 PM          93551 vista_sec.txt
-a-----        4/16/2026  12:56 PM         1215296 wazuh-agent.exe
-a-----        6/25/2026   2:03 PM           667 wazuh-agent.state
-a-----        6/25/2026   2:02 PM          1429 wazuh-logcollector.state
-a-----        4/16/2026  12:56 PM         310576 win32ui.exe
-a-----        4/16/2026  12:45 PM           367 win32ui.exe.manifest
-a-----        4/16/2026  12:45 PM          1367 wpk_root.pem

PS C:\Users\raoulbiga>

```

Figure 13
Installation agent Wazuh sur agent-windows via PowerShell.

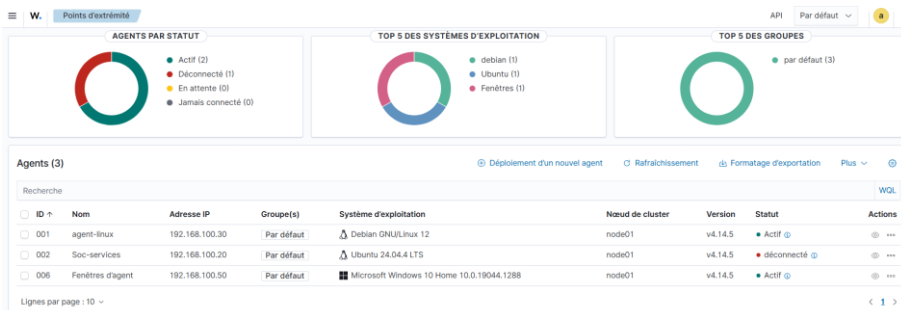


Figure 14
Dashboard Wazuh affichant 3 agents Active.

4.4 Phase 4 : Surveillance des Terminaux

Sur le wazuh-server, la surveillance des fichiers (FIM), Auditd et Sysmon ont été configurés via les règles locales. Une liste CDB a été créée pour identifier les programmes suspects. Les décodeurs ont été configurés pour interpréter correctement les événements. Sur agent-linux, Auditd a été activé pour journaliser les commandes exécutées. Sur agent-windows, Sysmon a été déployé avec une configuration de détection avancée.

```
[wazuh-user@wazuh-server ~]$ sudo cat /var/ossec/etc/lists/suspicious-programs
ncat:yellow
nc:red
netcat:red
tcpdump:orange
psexec:red
powershell:orange
cmd:red
wget:yellow
curl:yellow
ftp:yellow
tor.exe:red
mimikatz:red
[wazuh-user@wazuh-server ~]$ |
```

Figure 15
Liste CDB créée : /var/ossec/etc/lists/suspicious-programs.

```
[root@wazuh-server wazuh-user]# grep -E "rule id=\"1002|1003|1011\" /var/ossec/etc/rules/local_rules.xml | head -20
<rule id="100210" level="12">
<rule id="100200" level="7">
<rule id="100201" level="7">
<rule id="100300" level="5">
<rule id="100301" level="5">
<rule id="100302" level="5">
<rule id="100303" level="5">
<rule id="101100" level="5">
<rule id="101101" level="5">
<rule id="101103" level="5">
<rule id="101106" level="5">
<rule id="101107" level="5">
<rule id="101108" level="5">
<rule id="101110" level="5">
<rule id="101111" level="5">
<rule id="101115" level="5">
[root@wazuh-server wazuh-user]#
```

Figure 16
Règles FIM, Auditd et Sysmon dans `local_rules.xml`.

FIM : Événements récents					
Temps ↓	Chemin	Action	Description de la règle	Nivea...	Ident...
25 juin 2026 @ 07:59:32.706	/etc/resolv.conf.d/client-new.2568	supprimé	Fichier supprimé.	7	553
25 juin 2026 @ 07:59:32.706	/etc/resolv.conf	Modifié	La somme de contrôle d'intégrité a changé.	7	550
25 juin 2026 @ 07:59:32.663	/etc/resolv.conf.d/client-new.2568	ajouté	Fichier ajouté au système.	5	554
24 juin 2026 @ 15:08:20.076	/maison/test_fim.txt	supprimé	Fichier supprimé.	7	553
24 juin 2026 @ 15:08:20.027	/maison/test_fim.txt	ajouté	Fichier ajouté au système.	5	554

Figure 17
Événement FIM détecté dans le Dashboard Wazuh.

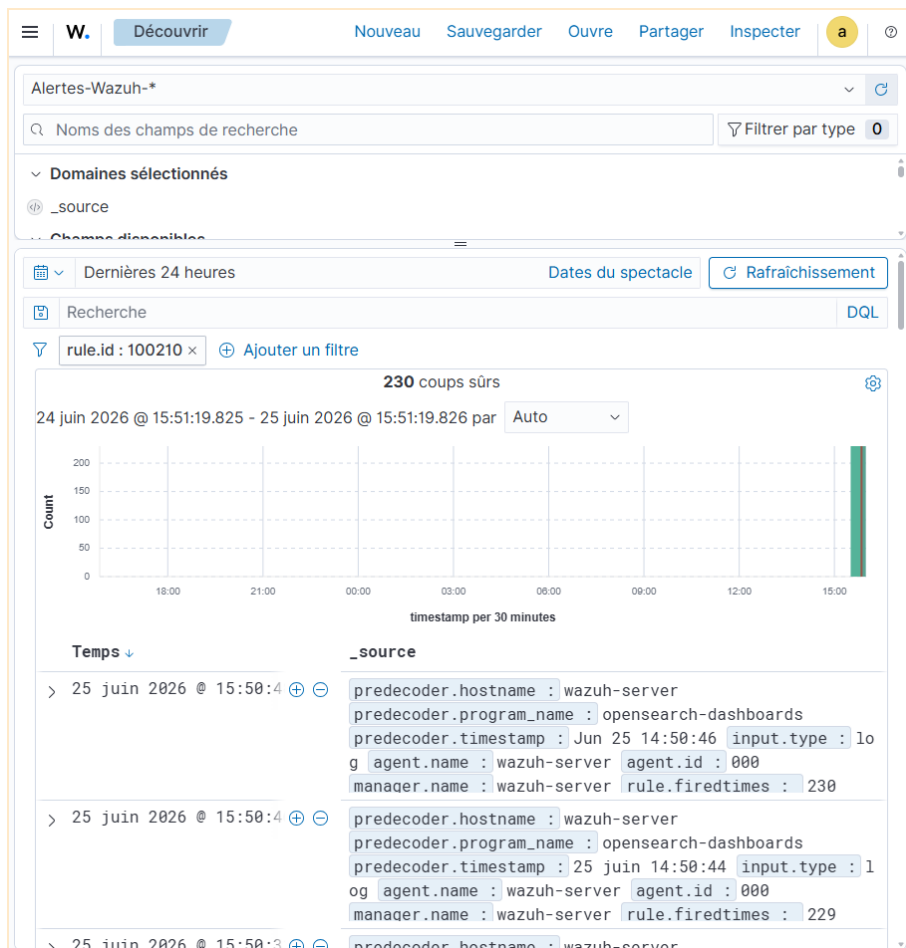


Figure 18
Alerte Auditd de commande suspecte (niveau 12).

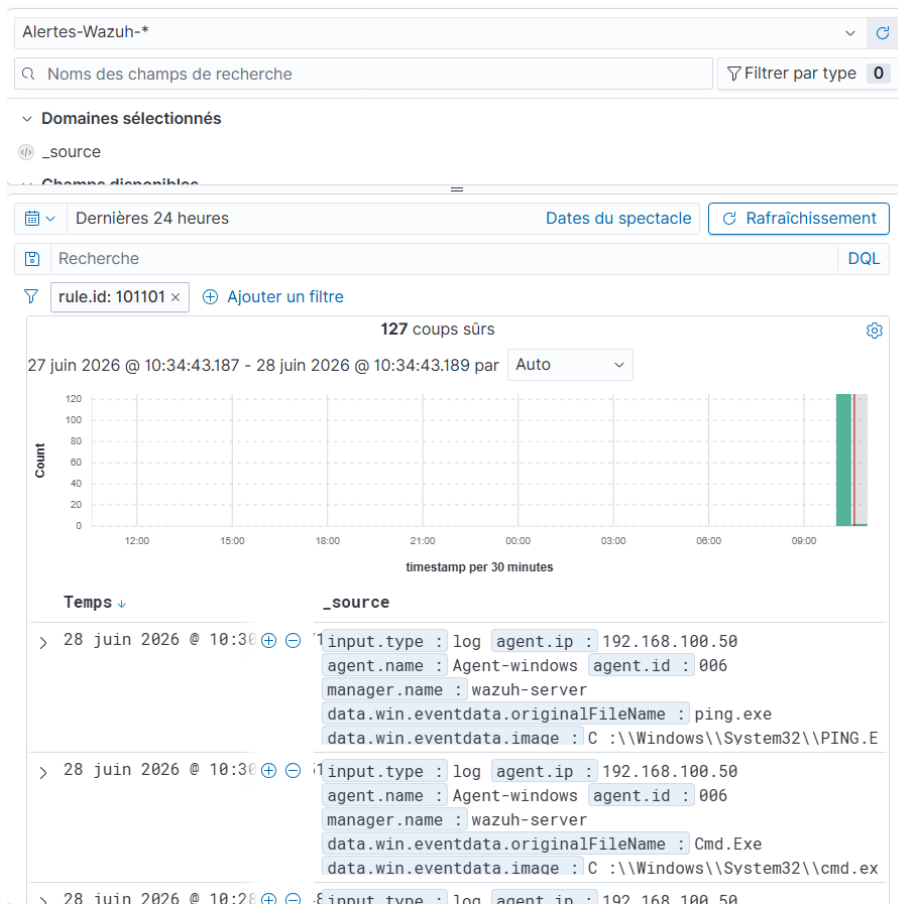


Figure 19
Alerte Audit de commande suspecte (niveau 12).

4.5 Phase 5 : Détection Réseau avec Suricata

Suricata a été installé directement sur la VM wazuh-server (Amazon Linux 2023) via dnf. Les règles Emerging Threats ont été téléchargées et configurées. L'interface réseau (eth0 ou enp0s3) a été spécifiée dans suricata.yaml pour la capture de trafic. Les alertes Suricata (au format JSON, eve.json) sont lues par l'agent Wazuh et remontées au serveur central.

Figure 20

Installation Suricata sur wazuh-server (dnf install).

```
[wazuh-user@wazuh-server ~]$ sudo grep -E "HOME_NET|af-packet|interface" /etc/suricata/suricata.yaml
HOME_NET: "192.168.100.0/24"
HTTP_SERVERS: "$HOME_NET"
SQL_SERVERS: "$HOME_NET"
SMTP_SERVERS: "$HOME_NET"
af-packet:
- interface: eth2
[wazuh-user@wazuh-server ~]$ |
```

Figure 21

Configuration suricata.yaml avec HOME_NET et interface réseau.

```
[wazuh-user@wazuh-server ~]$ ls -la /etc/suricata/rules/ | head -15
total 44888
drwxr-xr-x. 2 root root    16384 Jun 24 18:36 .
drwxr-xr-x. 3 root root     93 Jun 25 08:22 ..
-rw-r-----. 1 root root   2114 Jun 23 20:46 botcc.portgrouped.rules
-rw-r-----. 1 root root   4711 Jun 23 20:46 botcc.rules
-rw-r-----. 1 root root 334434 Jun 23 20:46 ciarmy.rules
-rw-r-----. 1 root root  20131 Jun 23 20:46 compromised.rules
-rw-r-----. 1 root root  53957 Jun 23 20:46 drop.rules
-rw-r-----. 1 root root   2666 Jun 23 20:46 dshield.rules
-rw-r-----. 1 root root 369791 Jun 23 20:46 emerging-activex.rules
-rw-r-----. 1 root root 646200 Jun 23 20:46 emerging-adware_pup.rules
-rw-r-----. 1 root root 656808 Jun 23 20:46 emerging-attack_response.rules
-rw-r-----. 1 root root   35976 Jun 23 20:46 emerging-chat.rules
-rw-r-----. 1 root root   20155 Jun 23 20:46 emerging-coinminer.rules
-rw-r-----. 1 root root 140532 Jun 23 20:46 emerging-current_events.rules
[wazuh-user@wazuh-server ~]$ |
```

Figure 22

Téléchargement des règles Emerging Threats.

```
[wazuh-user@wazuh-server ~]$ ps aux | grep suricata | grep -v grep
root      18776 55.5  5.2 872412 427880 ?        Ssl  09:49   0:15 suricata -c /etc/suricata/suricata.ya
ml -i eth2 -D
[wazuh-user@wazuh-server ~]$ |
```

Figure 23

Suricata actif et générant des alertes (systemctl status).

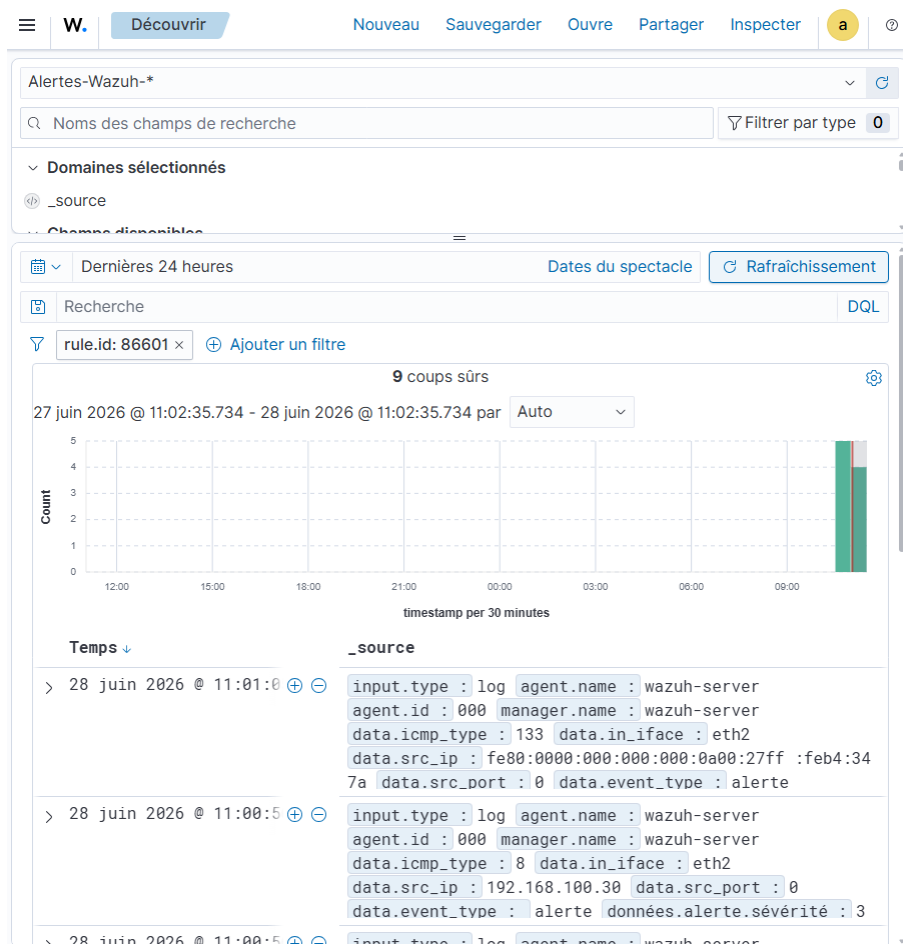


Figure 24
Alertes Suricata visibles dans le Dashboard Wazuh.

```
[root@wazuh-server wazuh-user]# sudo grep '"id": "86601"' /var/ossec/logs/alerts/alerts.json | grep "192.168.100.40" | tail -1 | jq -r '"Timestamp: \(.timestamp)\nSource IP: \(.data.src_ip)\nDestination IP: \(.data.dest_ip)\nSignature: \(.data.alert.signature)\nRule ID: \(.rule.id)\nLevel: \(.rule.level)'"
Timestamp: 2026-06-29T14:42:53.054+0000
Source IP: 192.168.100.40
Destination IP: 192.168.100.10
Signature: ET SCAN Potential SSH Scan OUTBOUND
Rule ID: 86601
Level: 3
[root@wazuh-server wazuh-user]#
```

Figure 25
Scan nmap depuis kali-attacker générant alertes Suricata.

4.6 Phase 6 : Analyse Malware avec YARA, VirusTotal et IA

La Phase 6 s'exécute sur trois VM différentes pour assurer une couverture complète. Sur agent-linux, YARA a été compilé à partir des sources (v4.5.1) et installé dans `/usr/local/bin`. Les règles Valhalla ont été téléchargées et configurées. Un script `yara.sh` a été créé pour automatiser la détection et l'enrichissement via l'API DeepSeek. Sur agent-windows, YARA a été installé via les binaires précompilés et un script `yara.py` a été créé pour une détection locale. Sur le wazuh-server, les décodeurs, règles, scripts de réponse active (`yara.sh` et `remove-threat.sh`) et l'intégration VirusTotal ont été configurés dans `ossec.conf` et `local_rules.xml`.

4.6.1 YARA sur Agent-Linux

YARA a été compilé et installé sur agent-linux avec les dépendances nécessaires (`make`, `gcc`, `autoconf`, etc.). Les règles Valhalla ont été téléchargées dans `/var/ossec/active-response/yara/rules/`. Le script `yara.sh` a été créé pour scanner automatiquement les fichiers détectés par FIM et soumettre les descriptions à l'API DeepSeek pour enrichissement contextuel.

```

drwxr-xr-x 3 root root 4096 25 juin 18:11 ..
-rw-r--r-- 1 root root 57788 25 juin 18:11 aclocal.m4
-rw-rw-r-- 1 root root 3008 25 mai 2024 appveyor.yml
-rw-rw-r-- 1 root root 569 25 mai 2024 AUTHORS
drwxr-xr-x 2 root root 4096 25 juin 18:11 autom4te.cache
drwxrwxr-x 2 root root 4096 25 mai 2024 bazel
-rw-rw-r-- 1 root root 75 25 mai 2024 .bazelrc
-rwxrwxr-x 1 root root 39 25 mai 2024 bootstrap.sh
drwxr-xr-x 2 root root 4096 25 juin 18:11 build-aux
-rw-rw-r-- 1 root root 4480 25 mai 2024 BUILD.bazel
-rwxrwxr-x 1 root root 41 25 mai 2024 build.sh
-rw-rw-r-- 1 root root 4370 25 mai 2024 .clang-format
drwxrwxr-x 3 root root 4096 25 mai 2024 cli
-rw-r--r-- 1 root root 43836 25 juin 18:11 config.log
-rwxr-xr-x 1 root root 56654 25 juin 18:11 config.status
-rwxr-xr-x 1 root root 555917 25 juin 18:11 configure
-rwxr-xr-x 1 root root 555917 16 juin 14:30 configure~
-rw-rw-r-- 1 root root 13746 25 mai 2024 configure.ac
-rw-rw-r-- 1 root root 1626 25 mai 2024 CONTRIBUTORS
-rw-rw-r-- 1 root root 1493 25 mai 2024 COPYING
drwxrwxr-x 2 root root 4096 25 mai 2024 dist
drwxrwxr-x 3 root root 4096 25 mai 2024 docs
drwxrwxr-x 3 root root 4096 25 mai 2024 extra
-rw-rw-r-- 1 root root 126 25 mai 2024 .gitattributes
drwxrwxr-x 4 root root 4096 25 mai 2024 .github
-rw-rw-r-- 1 root root 964 25 mai 2024 .gitignore
drwxr-xr-x 2 root root 4096 16 juin 14:32 .libs
-rwxr-xr-x 1 root root 348354 25 juin 18:11 libtool
drwxrwxr-x 9 root root 4096 25 mai 2024 libyara
-rw-r--r-- 1 root root 955 16 juin 14:32 libyara.la
drwxrwxr-x 2 root root 4096 25 juin 18:11 m4
-rw-r--r-- 1 root root 188392 25 juin 18:11 Makefile
-rw-rw-r-- 1 root root 10568 25 mai 2024 Makefile.am
-rw-r--r-- 1 root root 204872 25 juin 18:11 Makefile.in
-rw-rw-r-- 1 root root 6526 25 mai 2024 README.md
-rw-rw-r-- 1 root root 1028 25 mai 2024 .readthedocs.yaml
-rw-rw-r-- 1 root root 6 25 mai 2024 sample.file
-rw-rw-r-- 1 root root 179 25 mai 2024 sample.rules
drwxrwxr-x 2 root root 4096 25 mai 2024 sandbox
-rw-rw-r-- 1 root root 604 25 mai 2024 SECURITY.md
drwxrwxr-x 5 root root 4096 25 mai 2024 tests
drwxrwxr-x 5 root root 4096 25 mai 2024 windows
-rw-rw-r-- 1 root root 1970 25 mai 2024 WORKSPACE.bazel
-rwxr-xr-x 1 root root 6264 16 juin 14:32 yara
-rwxr-xr-x 1 root root 6274 16 juin 14:32 yarac
-rw-rw-r-- 1 root root 1676 25 mai 2024 yarac.man
-rw-rw-r-- 1 root root 4063 25 mai 2024 yara.man
-rw-r--r-- 1 root root 286 25 juin 18:11 yara.pc
-rw-rw-r-- 1 root root 323 25 mai 2024 yara.pc.in
raoulbiga@agent-linux:/usr/local/bin/yara-4.5.1$

```



```

fi

YARA_output="$(cat "${YARA_PATH}"/yara -w -r -m "$YARA_RULES" "$FILENAME")"

if [[ $YARA_output != "" ]]; then
    if rm -rf "$FILENAME"; then
        echo "wazuh-YARA: INFO - Successfully deleted $FILENAME" >> ${LOG_FILE}
    else
        echo "wazuh-YARA: INFO - Unable to delete $FILENAME" >> ${LOG_FILE}
    fi

    while read -r line; do
        description=$(echo "$line" | grep -oP '(?<=description=).*?(?=)')
        if [[ $description != "" ]]; then
            payload=$(jq -n \
                --arg desc "$description" \
                --arg model "$DEEPSEEK_MODEL" \
                '{
                    model: $model,
                    messages: [
                        {
                            role: "user",
                            content: "In one paragraph, tell me about the impact and how to mitigate \($desc)"
                        }
                    ],
                    max_tokens: 256
                }')

            deepseek_response=$(curl -s -X POST "https://api.deepseek.com/v1/chat/completions" \
                -H "Content-Type: application/json" \
                -H "Authorization: Bearer $API_KEY" \
                -d "$payload")

            response_text=$(echo "$deepseek_response" | jq -r '.choices[0].message.content')

            if [[ $response_text == "null" ]] || [[ -z "$response_text" ]]; then
                echo "wazuh-YARA: INFO - Scan result: $line | deepseek_response: none" >> ${LOG_FILE}
            else
                echo "wazuh-YARA: INFO - Scan result: $line | chatgpt_response: $response_text" >> ${LOG_FILE}
            fi
        else
            echo "wazuh-YARA: INFO - Scan result: $line" >> ${LOG_FILE}
        fi
    done <<< "$YARA_output"
else
    echo "wazuh-YARA: INFO - No YARA rule matched." >> ${LOG_FILE}
fi

exit 0
root@agent-linux:/usr/local/bin/yara-4.5.1#

```

Figure 29

Script yara.sh créé dans /var/ossec/active-response/bin/.

```

root@agent-linux:/usr/local/bin/yara-4.5.1# sudo grep -q "eicar_test" /var/ossec/active-response/yara/rules/yara_rules.yar || echo 'rule eicar_test {
    strings:
        $a = "X50!P%AP[4\\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!"
    condition:
        $a
}' | sudo tee -a /var/ossec/active-response/yara/rules/yara_rules.yar
rule eicar_test {
    strings:
        $a = "X50!P%AP[4\\PZX54(P^)7CC)7}$EICAR-STANDARD-ANTIVIRUS-TEST-FILE!"
    condition:
        $a
}
root@agent-linux:/usr/local/bin/yara-4.5.1# sudo yara -w -r -m /var/ossec/active-response/yara/rules/yara_rules.yar /tmp/eicar.txt
eicar_test [] /tmp/eicar.txt
root@agent-linux:/usr/local/bin/yara-4.5.1#

```

Figure 30

Test YARA manual sur un fichier EICAR.

4.6.2 YARA sur Agent-Windows

YARA pour Windows a été installé via les binaires précompilés (yara64.exe). Un script yara.py a été créé et compilé en yara.exe via PyInstaller pour lancer des scans automatiques sur les fichiers Windows suspects. Les fichiers yara.exe et les règles ont été placés dans C:\Program Files (x86)\ossec-agent\active-response\bin\yara\.

```
PS C:\Program Files\ossec-agent\active-response\bin> dir *.exe

Directory: C:\Program Files\ossec-agent\active-response\bin

Mode                LastWriteTime         Length Name
----                -
-a---             4/16/2026 12:55 PM         206000 netsh.exe
-a---             4/16/2026 12:56 PM         199856 restart-wazuh.exe
-a---             4/16/2026 12:56 PM         202416 route-null.exe

PS C:\Program Files\ossec-agent\active-response\bin> cd "C:\Program Files\ossec-agent\active-response\bin\yara"
PS C:\Program Files\ossec-agent\active-response\bin\yara> dir

Directory: C:\Program Files\ossec-agent\active-response\bin\yara

Mode                LastWriteTime         Length Name
----                -
d----             6/24/2026 12:54 PM              rules
-a---             5/25/2024  2:24 PM        1663488 yara.exe

PS C:\Program Files\ossec-agent\active-response\bin\yara> .\yara --version
4.5.1
PS C:\Program Files\ossec-agent\active-response\bin\yara>
```

Figure 31

Installation binaire YARA (yara64.exe) sur agent-windows.

```

Administrator: PowerShell 7 (x86)
PS C:\Program Files\ossec-agent\active-response\bin\yara> cd "C:\Program Files\ossec-agent\active-response\bin"
PS C:\Program Files\ossec-agent\active-response\bin> type yara.py
import os
import subprocess
import json
import re
import requests

API_KEY = 'sk-83b2f0e2ad5e441899a21bca8afe3b'
DEEPSEEK_MODEL = 'deepseek-chat'

if os.environ['PROCESSOR_ARCHITECTURE'].endswith('86'):
    log_file_path = os.path.join(os.environ['ProgramFiles'], 'ossec-agent', 'active-response', 'active-responses.log')
else:
    log_file_path = os.path.join(os.environ['ProgramFiles(x86)'], 'ossec-agent', 'active-response', 'active-responses.log')

def log_message(message):
    with open(log_file_path, 'a') as log_file:
        log_file.write(message + '\n')

def read_input():
    return input()

def get_syscheck_file_path(json_file_path):
    with open(json_file_path, 'r') as json_file:
        data = json.load(json_file)
    return data['parameters']['alert']['syscheck']['path']

def run_yara_scan(yara_exe_path, yara_rules_path, syscheck_file_path):
    try:
        result = subprocess.run([yara_exe_path, '-m', yara_rules_path, syscheck_file_path], capture_output=True, text=True)
        return result.stdout.strip()
    except Exception as e:
        log_message(f"Error running Yara scan: {str(e)}")
        return None

def extract_description(yara_output):
    match = re.search(r'description="([^\"]+)"', yara_output)
    return match.group(1) if match else None

def query_deepseek(description):
    headers = {
        'Authorization': f'Bearer {API_KEY}',
        'Content-Type': 'application/json'
    }
    data = {
        'model': DEEPSEEK_MODEL,
        'messages': [{ 'role': 'user', 'content': f'In one paragraph, tell me about the impact and how to mitigate {description}' }],
        'max_tokens': 256
    }
    response = requests.post('https://api.deepseek.com/v1/chat/completions', headers=headers, json=data)

```

Figure 32

Script yara.py créé pour agent-windows.

```

Administrator: PowerShell 7 (x86)
PS C:\Program Files\ossec-agent\active-response\bin> cd "C:\Program Files\ossec-agent\active-response\bin\yara"
PS C:\Program Files\ossec-agent\active-response\bin\yara> dir yara.exe

Directory: C:\Program Files\ossec-agent\active-response\bin\yara

Mode                LastWriteTime         Length Name
----                -
-a---             5/25/2024  2:24 PM        1663488 yara.exe

PS C:\Program Files\ossec-agent\active-response\bin\yara>

```

Figure 33

Compilation yara.py en yara.exe via PyInstaller.

```

PS C:\Program Files\ossec-agent\active-response\bin\yara> echo 'rule all_files { condition: true }' | Out-File -FilePath all.yar -Encoding ascii
PS C:\Program Files\ossec-agent\active-response\bin\yara> .\yara.exe all.yar C:\temp\test.txt
all_files C:\temp\test.txt
PS C:\Program Files\ossec-agent\active-response\bin\yara> echo "test" | Out-File -FilePath C:\temp\test2.txt -Encoding ascii -NoNewLine
PS C:\Program Files\ossec-agent\active-response\bin\yara> .\yara.exe -w -r -m test_simple.yar C:\temp\test2.txt
test_rule [] C:\temp\test2.txt
PS C:\Program Files\ossec-agent\active-response\bin\yara> |

```

Figure 34

Test YARA manual sur Windows (yara64.exe sur fichier test).

4.6.3 Décodeurs, Règles et Réponse Active sur Wazuh

Sur le wazuh-server, les décodeurs locaux ont été créés pour parser les sorties YARA. Les règles (local_rules.xml) ont été configurées pour déclencher des alertes lors de détections YARA positives. Deux scripts de réponse active ont été créés : yara.sh pour exécuter YARA selon les règles du serveur, et remove-threat.sh pour supprimer les fichiers malveillants confirmés par VirusTotal. L'intégration VirusTotal dans ossec.conf automatise la vérification de réputation des fichiers.

```
[wazuh-user@wazuh-server ~]$ sudo cat /var/ossec/etc/decoders/local_decoder.xml
<decoder name="YARA_decoder">
  <prematch>wazuh-YARA:</prematch>
</decoder>
<decoder name="YARA_child">
  <parent>YARA_decoder</parent>
  <regex type="pcre2">wazuh-YARA: (\S+)</regex>
  <order>YARA.log_type</order>
</decoder>
<decoder name="YARA_child">
  <parent>YARA_decoder</parent>
  <regex type="pcre2">Scan result: (\S+)\s+</regex>
  <order>YARA.rule_name</order>
</decoder>
<decoder name="YARA_child">
  <parent>YARA_decoder</parent>
  <regex type="pcre2">\[description="([^\"]+)"</regex>
  <order>YARA.rule_description</order>
</decoder>
<decoder name="YARA_child">
  <parent>YARA_decoder</parent>
  <regex type="pcre2">\] (.*) \]</regex>
  <order>YARA.scanned_file</order>
</decoder>
<decoder name="YARA_child">
  <parent>YARA_decoder</parent>
  <regex type="pcre2">chatgpt_response: (.*)</regex>
  <order>YARA.chatgpt_response</order>
</decoder>
<decoder name="YARA_child">
  <parent>YARA_decoder</parent>
  <regex type="pcre2">Successfully deleted (.*)</regex>
  <order>YARA.file_deleted</order>
</decoder>
<decoder name="YARA_child">
  <parent>YARA_decoder</parent>
  <regex type="pcre2">Unable to delete (.*)</regex>
  <order>YARA.file_not_deleted</order>
</decoder>
<decoder name="virustotal_decoder">
  <prematch>wazuh-VIRUSTOTAL:</prematch>
</decoder>
<decoder name="virustotal_child">
  <parent>virustotal_decoder</parent>
  <regex type="pcre2">wazuh-VIRUSTOTAL: (\S+) - (.*)</regex>
  <order>VT.level, VT.message</order>
</decoder>
[wazuh-user@wazuh-server ~]$ |
```

Figure 35

Décodeurs YARA dans /var/ossec/etc/decoders/local_decoder.xml.

```
[wazuh-user@wazuh-server ~]$ sudo grep -E "108000|108001|108002|108003" /var/ossec/etc/rules/local_rules.xml
<rule id="108000" level="0">
<rule id="108001" level="10">
  <if_sid>108000</if_sid>
<rule id="108002" level="5">
  <if_sid>108000</if_sid>
<rule id="108003" level="12">
  <if_sid>108000</if_sid>
[wazuh-user@wazuh-server ~]$
```

Figure 36

Règles YARA (108000-108003) dans local_rules.xml.

```
[root@wazuh-server wazuh-user]# ls -l /var/ossec/active-response/bin/remove-threat.sh
-rwxr-x---. 1 root wazuh 0 Jun 25 16:16 /var/ossec/active-response/bin/remove-threat.sh
[root@wazuh-server wazuh-user]#
```

Figure 37

Script yara.sh créé sur wazuh-server avec permissions root:wazuh.

```
[root@wazuh-server wazuh-user]# ls -l /var/ossec/active-response/bin/remove-threat.sh
-rwxr-x---. 1 root wazuh 0 Jun 25 16:16 /var/ossec/active-response/bin/remove-threat.sh
[root@wazuh-server wazuh-user]#
```

Figure 38

Script remove-threat.sh pour suppression automatique malware.

```
[root@wazuh-server wazuh-user]# sudo grep -A 8 "virustotal" /var/ossec/etc/ossec.conf | head -10
<name>custom-virustotal</name>
<level>3</level>
<alert_format>json</alert_format>
<rule_id>101100,101103,550,554,87105,108000,108001,80792,5710,86601</rule_id>
</integration>

<!-- ===== INTÉGRATION VIRUSTOTAL ===== -->
<integration>
  <name>virustotal</name>
  <api_key>bfe1615376115042af280354bb6a502526e15cfae79f6e8997b548f12a71eda3</api_key>
[root@wazuh-server wazuh-user]#
```

Figure 39

Intégration VirusTotal configurée dans ossec.conf.

```
[root@wazuh-server wazuh-user]# sudo tail -f /var/ossec/logs/alerts/alerts.json | grep -E '100300|108001'
{"timestamp":"2026-06-29T05:04:59.624+0000","rule":{"level":10,"description":"YARA: Fichier \"\" correspon
d à eicar_test","id":"108001","firedtimes":1,"mail":false,"groups":["local"],"agent":{"id":"001","name":
"agent-linux","ip":"192.168.100.30"},"manager":{"name":"wazuh-server"},"id":"1782709499.396910"},"full
_log":"wazuh-YARA: INFO - Scan result: eicar_test [] /home/raoulbiga/eicar_final3.txt","decoder":{"name":
"YARA_decoder"},"data":{"YARA":{"log_type":"INFO","rule_name":"eicar_test"},"location":"/var/ossec/log
s/active-responses.log"}}
{"timestamp":"2026-06-29T05:05:03.609+0000","rule":{"level":10,"description":"YARA: Fichier \"\" correspon
d à eicar_test","id":"108001","firedtimes":2,"mail":false,"groups":["local"],"agent":{"id":"001","name":
"agent-linux","ip":"192.168.100.30"},"manager":{"name":"wazuh-server"},"id":"1782709503.398288"},"full
_log":"wazuh-YARA: INFO - Scan result: eicar_test [] /home/raoulbiga/eicar_final3.txt","decoder":{"name":
"YARA_decoder"},"data":{"YARA":{"log_type":"INFO","rule_name":"eicar_test"},"location":"/var/ossec/log
s/active-responses.log"}}
{"timestamp":"2026-06-29T05:06:40.199+0000","rule":{"level":10,"description":"YARA: Fichier \"\" correspon
d à eicar_test","id":"108001","firedtimes":3,"mail":false,"groups":["local"],"agent":{"id":"001","name":
"agent-linux","ip":"192.168.100.30"},"manager":{"name":"wazuh-server"},"id":"1782709600.401212"},"full
_log":"wazuh-YARA: INFO - Scan result: eicar_test [] /home/raoulbiga/eicar_final3.txt","decoder":{"name":
"YARA_decoder"},"data":{"YARA":{"log_type":"INFO","rule_name":"eicar_test"},"location":"/var/ossec/log
s/active-responses.log"}}
```

Figure 40

Alerte 108001 affichant détection YARA positive.

```
[root@wazuh-server wazuh-user]# sudo grep '"id":108002"' /var/ossec/logs/alerts/alerts.json | tail -1 |
python3 -m json.tool
{
  "timestamp": "2026-06-29T05:06:40.194+0000",
  "rule": {
    "level": 5,
    "description": "YARA: Fichier malveillant supprim\u00e9: /home/raulbiga/eicar_final3.txt",
    "id": "108002",
    "firedtimes": 3,
    "mail": false,
    "groups": [
      "local"
    ]
  },
  "agent": {
    "id": "001",
    "name": "agent-linux",
    "ip": "192.168.100.30"
  },
  "manager": {
    "name": "wazuh-server"
  },
  "id": "1782709600.400854",
  "full_log": "wazuh-YARA: INFO - Successfully deleted /home/raulbiga/eicar_final3.txt",
  "decoder": {
    "name": "YARA_decoder"
  },
  "data": {
    "YARA": {
      "log_type": "INFO",
      "file_deleted": "/home/raulbiga/eicar_final3.txt"
    }
  },
  "location": "/var/ossec/logs/active-responses.log"
}
[root@wazuh-server wazuh-user]#
```

Figure 41

Alerte 108002 confirmant suppression automatique du fichier.

```
root@agent-linux:/home/raulbiga# sudo grep -i "chatgpt_response" /var/ossec/logs/active-responses.log |
tail -2
wazuh-YARA: INFO - chatgpt_response: Test DeepSeek
root@agent-linux:/home/raulbiga#
```

Figure 42

Enrichissement DeepSeek visible dans le champ YARA.chatgpt_response.

4.7 Phase 7 : Threat Intelligence avec MISP

MISP a été déployé via Docker sur soc-services (port 1443). Un script Python custom-misp.py a été créé sur wazuh-server pour interroger l'API MISP lors de chaque alerte pertinente. Le script extrait des IOCs (adresses IP, domaines, hachés de fichiers) et vérifie leur présence dans la base MISP. Les résultats sont réinjectés dans Wazuh sous forme d'alertes enrichies pour signaler les menaces connues.

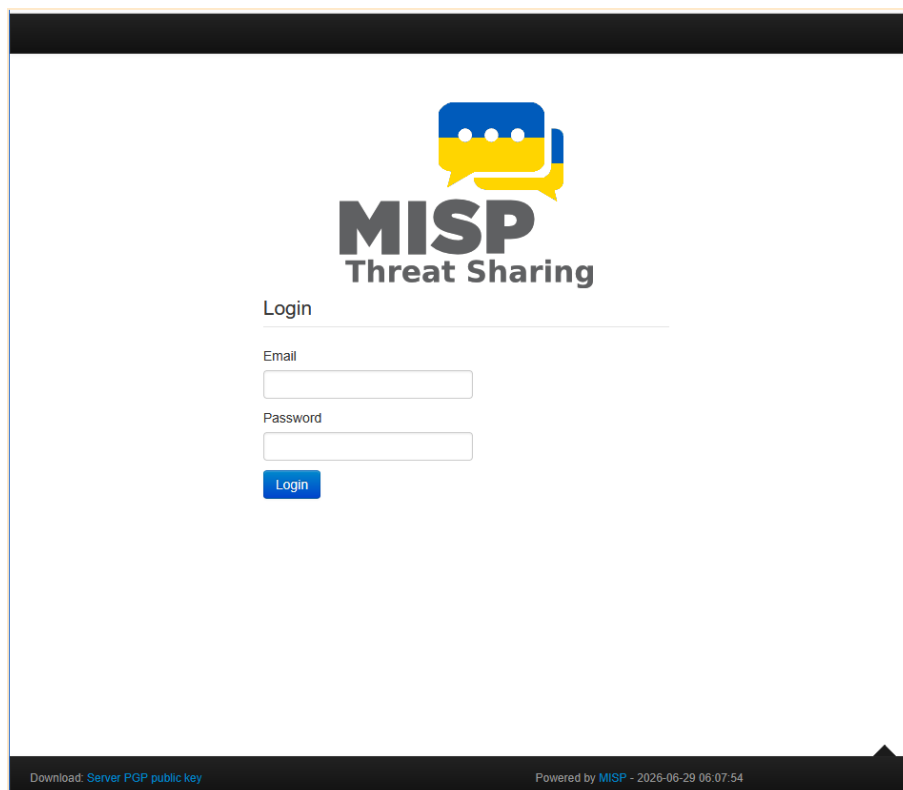
```

root@soc-services:/home/naoulbiga# docker ps | grep misp
f80f00f28cbf   ghcr.io/misp/misp-docker/misp-core:latest   "/entrypoint.sh"   6 days ago   Up 2 days (unhealth
0.0.0.0:443->443/tcp, [::]:443->443/tcp
3f2ffcdcd63    mariadb:10.11                                "docker-entrypoint.s..."   6 days ago   Up 2 days (healthy)
misp-docker-misp-co
8b4f990afb79   valkey/valkey:7.2                            "docker-entrypoint.s..."   6 days ago   Up 2 days (healthy)
misp-docker-db-1
33a52ca59abd   ghcr.io/misp/misp-docker/misp-modules:latest "/usr/local/bin/misp..."   6 days ago   Up 2 days (healthy)
misp-docker-redis-1
0d0b3e43d30c   ghcr.io/egos-tech/smtp:1.1.3                "/bin/entrypoint.sh ..."   10 days ago   Up 2 days
misp-docker-misp-mc
misp-docker-mail-1
root@soc-services:/home/naoulbiga# _

```

Figure 43

Déploiement MISP via Docker sur soc-services.



The screenshot shows the MISP Threat Sharing web interface. At the top, there is a logo consisting of a blue speech bubble with three white dots and a yellow speech bubble below it. Below the logo, the text "MISP Threat Sharing" is displayed in a bold, sans-serif font. Underneath, the word "Login" is centered. Below "Login", there are two input fields: "Email" and "Password". Below the "Password" field, there is a blue "Login" button. At the bottom of the page, there is a footer with two links: "Download: Server PGP public key" and "Powered by MISP - 2026-06-29 06:07:54".

Figure 44

Interface web MISP accessible via <https://192.168.100.20:1443>.

```

if len(sys.argv) < 2:
    log("No alert file")
    return
try:
    with open(sys.argv[1]) as f:
        alert = json.load(f)
except Exception as e:
    log(f"Cannot read alert: {e}")
    return

rule_id = alert.get('rule', {}).get('id', 'unknown')
log(f"Processing Rule: {rule_id}")

if rule_id not in RULE_CONFIG:
    log(f"Rule {rule_id} not configured")
    return

config = RULE_CONFIG[rule_id]

# Pour les règles YARA, on extrait le chemin depuis le full_log
if rule_id in ["108000", "108001"]:
    full_log = alert.get('full_log', '')
    ioc_value = extract_yara_path(full_log)
    if not ioc_value:
        # Fallback sur le champ syscheck.path (si présent)
        ioc_value = get_nested(alert, config["field"])
else:
    ioc_value = get_nested(alert, config["field"])

if not ioc_value:
    log(f"No IOC found for rule {rule_id}")
    return

log(f"IOC: {str(ioc_value)[:100]}")
result = check_misp(str(ioc_value), config["type"])
if result.get("found"):
    log(f"IOC FOUND - Event: {result.get('event_id')}")
    out = {"misp": {"source": {"description": alert.get('rule', {}).get('description', ''), "type": config["source"]},
                    "event_id": result.get("event_id"), "value": str(ioc_value),
                    "type": config["type"], "status": "ioc_found", "integration": "misp"}
    send_event(out, alert.get("agent"))
elif "error" in result:
    log(f"MISP error: {result['error']}")
else:
    log("IOC NOT FOUND in MISP")

if __name__ == "__main__":
    main()
[root@wazuh-server wazuh-user]#

```

Figure 45

Script custom-misp.py créé sur wazuh-server.

```

[root@wazuh-server wazuh-user]# sudo grep -A 8 "custom-misp" /var/ossec/etc/ossec.conf
<name>custom-misp</name>
<level>5</level>
<alert_format>json</alert_format>
<rule_id>101100,101103,550,554,87105,108000,108001,80792,5710,86601</rule_id>
</integration>

<!-- ===== INTEGRATION IRIS ===== -->
<integration>
  <name>custom-wazuh_iris</name>
[root@wazuh-server wazuh-user]#

```

Figure 46

Intégration MISP dans ossec.conf.


```
[root@wazuh-server wazuh-user]# sudo tail -20 /var/ossec/logs/misp-integration.log
[2026-06-29 06:01:23] Processing Rule: 550
[2026-06-29 06:01:23] IOC: f1283dadae7c65a24852623282b6ec235a83e32dfa53c75745de98a672791d41
[2026-06-29 06:01:32] IOC NOT FOUND in MISP
[2026-06-29 06:02:00] Processing Rule: 550
[2026-06-29 06:02:00] IOC: 4307aa7cc97a4db32a674ad32f893b251188903cafa6d5266c813fc5c9ea755e
[2026-06-29 06:02:11] MISP API error: HTTPConnectionPool(host='192.168.100.20', port=443): Read timed out. (read timeout=10)
[2026-06-29 06:02:11] MISP error: HTTPConnectionPool(host='192.168.100.20', port=443): Read timed out. (read timeout=10)
[2026-06-29 06:02:23] Processing Rule: 550
[2026-06-29 06:02:23] IOC: 9ee6feef0d2f23a30414eeb5152b0ab5178fbf80c3ef2094e9e4552c72e7e298
[2026-06-29 06:02:34] MISP API error: HTTPConnectionPool(host='192.168.100.20', port=443): Read timed out. (read timeout=10)
[2026-06-29 06:02:34] MISP error: HTTPConnectionPool(host='192.168.100.20', port=443): Read timed out. (read timeout=10)
[2026-06-29 06:36:34] Processing Rule: 554
[2026-06-29 06:36:34] IOC: e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
[2026-06-29 06:36:39] IOC NOT FOUND in MISP
[2026-06-29 06:36:42] Processing Rule: 550
[2026-06-29 06:36:42] IOC: 725deedb14d7d72329f4b9bc8346c2581c0ad4d1a9b97e865c7080a912f11c1e
[2026-06-29 06:36:46] IOC NOT FOUND in MISP
[2026-06-29 06:37:02] Processing Rule: 550
[2026-06-29 06:37:02] IOC: 725deedb14d7d72329f4b9bc8346c2581c0ad4d1a9b97e865c7080a912f11c1e
[2026-06-29 06:37:08] IOC NOT FOUND in MISP
[root@wazuh-server wazuh-user]#
```

Figure 47

Alerte 100622 signalant détection IOC malveillant via MISP.

4.8 Phase 8 : Gestion des Incidents avec DFIR-IRIS

DFIR-IRIS a été déployé via Docker sur soc-services (port 8443). Un script Python custom-wazuh_iris.py a été créé sur wazuh-server pour transmettre automatiquement les alertes Wazuh à IRIS via webhook. Chaque alerte crée un cas d'incident structuré dans IRIS, accessible aux analystes pour investigation et suivi.

```
root@soc-services:/home/raoulbiga# docker ps | grep iris
f1933ed87ede ghcr.io/dfir-iris/iriswebapp_app:latest      "/wait-for-irisweba..." 2 days ago Up
54 minutes
67c84afd4d98 ghcr.io/dfir-iris/iriswebapp_nginx:latest      "/entrypoint.sh"         2 days ago Up
54 minutes (healthy) 80/tcp, 0.0.0.0:8443->8443/tcp, [::]:8443->8443/tcp
3b0c291b89b5 ghcr.io/dfir-iris/iriswebapp_app:latest      "nohup ./iris-entryp..." 2 days ago Up
54 minutes
bfff3dd6ee26b rabbitmq:3-management-alpine      "docker-entrypoint.s..." 2 days ago Up
54 minutes 4369/tcp, 5671-5672/tcp, 15671-15672/tcp, 15691-15692/tcp, 25672/tcp
05483f165748 ghcr.io/dfir-iris/iriswebapp_db:latest      "docker-entrypoint.s..." 2 days ago Up
54 minutes 5432/tcp
root@soc-services:/home/raoulbiga#
```

Figure 48

Déploiement IRIS via Docker sur soc-services.

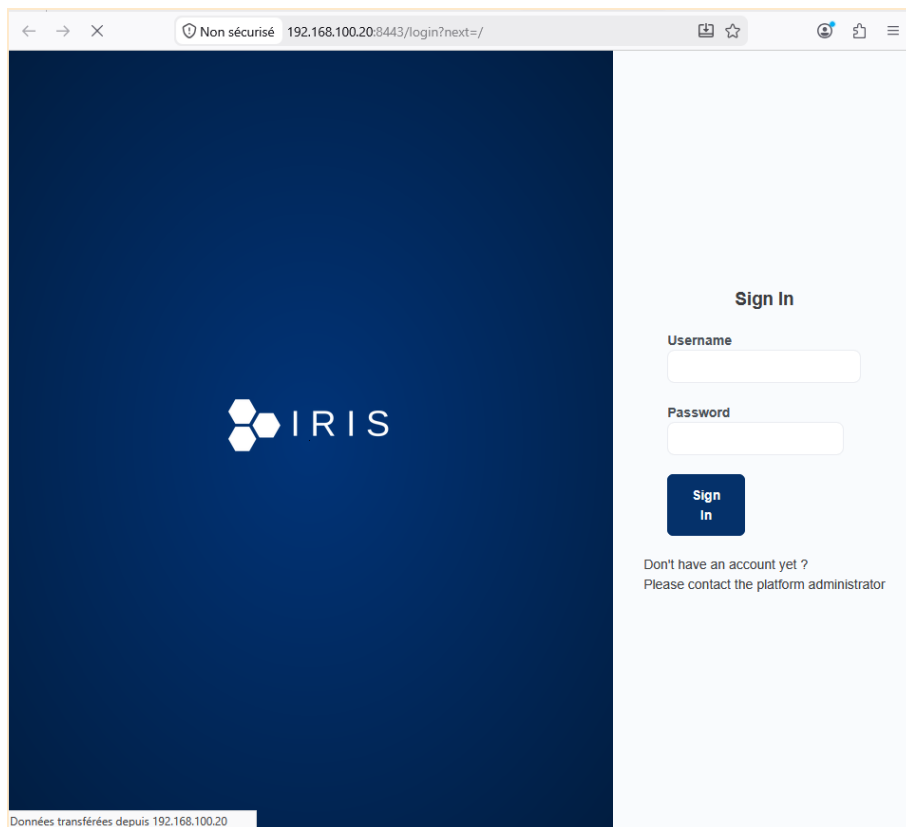


Figure 49
Interface IRIS accessible via <https://192.168.100.20:8443>.

```

        "alert_severity_id": 3 if level < 5 else 4 if level < 7 else 5,
        "alert_status_id": 2,          # "New"
        "alert_customer_id": 1,
        "alert_source": "Wazuh",
        "alert_source_event_time": timestamp,
        "alert_tags": "wazuh," + agent_name,
        "alert_note": "",
        "alert_source_content": alert
    }

def send_to_iris(payload):
    headers = {
        "Authorization": f"Bearer {IRIS_API_KEY}",
        "Content-Type": "application/json"
    }
    try:
        r = requests.post(
            IRIS_API_URL,
            json=payload,
            headers=headers,
            verify=IRIS_VERIFY_SSL,
            timeout=10
        )
        if r.status_code in [200, 201, 202]:
            logging.info(f"Alerte envoyée à IRIS (HTTP {r.status_code})")
            return True
        else:
            logging.error(f"IRIS HTTP {r.status_code}: {r.text[:200]}")
            return False
    except Exception as e:
        logging.error(f"Erreur connexion IRIS: {e}")
        return False

def main():
    if len(sys.argv) < 2:
        logging.error("Aucun fichier d'alerte fourni")
        sys.exit(1)
    try:
        with open(sys.argv[1]) as f:
            alert = json.load(f)
    except Exception as e:
        logging.error(f"Lecture alerte: {e}")
        sys.exit(1)
    payload = format_alert(alert)
    success = send_to_iris(payload)
    sys.exit(0 if success else 1)

if __name__ == "__main__":
    main()
[root@wazuh-server wazuh-user]# cat /var/ossec/integrations/custom-wazuh_iris.py

```

Figure 50

Script custom-wazuh_iris.py créé sur wazuh-server.

```

[root@wazuh-server wazuh-user]# sudo grep -A 10 "custom-wazuh_iris" /var/ossec/etc/ossec.conf
<name>custom-wazuh_iris</name>
<hook_url>https://192.168.100.20:8443/webhooks/wazuh</hook_url>
<level>3</level>
<rule_id>100200,100210,100300,100301,108001,108002,101100,101103,101115</rule_id>
<options>alert_format=json</options>
</integration>

<!-- ===== ACTIVE RESPONSE YARA ===== -->
<command>
  <name>yara_linux</name>
  <executable>yara.sh</executable>
[root@wazuh-server wazuh-user]# |

```

Figure 51

Intégration IRIS dans ossec.conf (webhook).

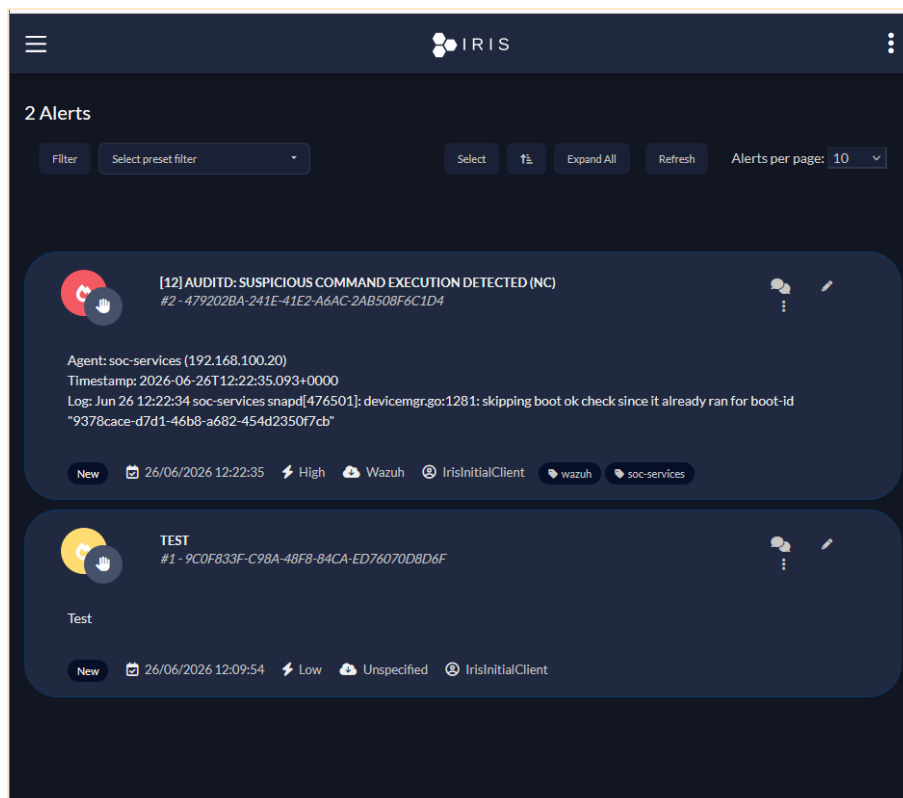


Figure 52
Cas d'incident créé automatiquement dans IRIS.

4.9 Phase 9 : Automatisation et Notification Telegram

Shuffle a été déployé via Docker sur soc-services (port 3001). Un workflow a été configuré pour extraire les IOCs détectés, les vérifier sur VirusTotal, et envoyer les résultats sur Telegram via l'API TelegramBot. Un script Python custom-telegram.py a été créé sur wazuh-server pour envoyer directement les alertes critiques sur Telegram. Le numéro de conversation et la clé API TelegramBot ont été configurés dans le script.

```

root@soc-services:/home/raoulbiga# docker ps | grep shuffle
0f0dfe6b3b58  ghcr.io/shuffle/shuffle-frontend:latest  "/entrypoint.sh nginx..."  2 days ago  Up
About an hour  0.0.0.0:3001->80/tcp, [::]:3001->80/tcp, 0.0.0.0:3443->443/tcp, [::]:3443->4
43/tcp  shuffle-frontend
f069b0e4a095  ghcr.io/shuffle/shuffle-orborus:latest  "/orborus"  2 days ago  Up
About an hour  shuffle-orborus
4875e7e702cb  memcached:alpine  "docker-entrypoint.s..."  2 days ago  Up
About an hour  0.0.0.0:11211->11211/tcp, [::]:11211->11211/tcp  shuffle-memcached
c8adcfla4c4e  ghcr.io/shuffle/shuffle-backend:latest  "/shufflebackend"  2 days ago  Up
About an hour  0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp  shuffle-backend
root@soc-services:/home/raoulbiga#

```

Figure 53

Déploiement Shuffle via Docker sur soc-services.

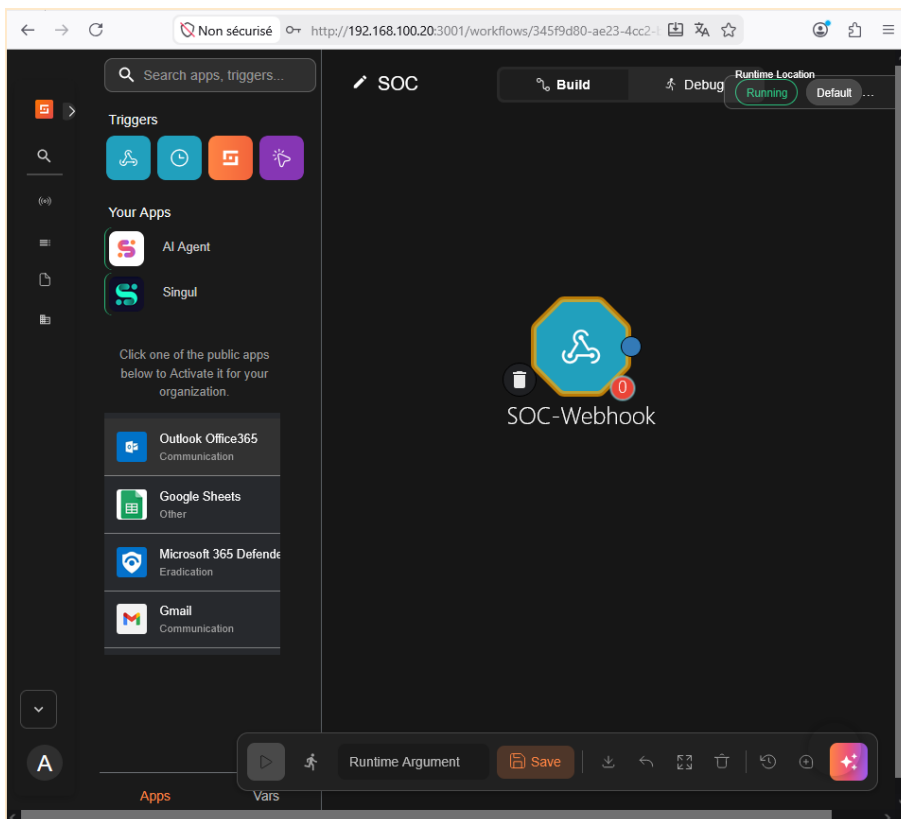


Figure 54

Workflow Shuffle configuré pour extraction IOC et VirusTotal.

```
[root@wazuh-server wazuh-user]# sudo grep -A 8 "shuffle" /var/ossec/etc/ossec.conf
  <name>shuffle</name>
  <hook_url>http://192.168.100.20:3001/api/v1/hooks/webhook_354bd8a7-1f7e-4312-93e5-cf310eb98d1d</hook
_url>
  <level>10</level>
  <alert_format>json</alert_format>
  <group>authentication_failed,malware,web_attack,vulnerability</group>
</integration>

<integration>
  <name>custom-telegram</name>
[root@wazuh-server wazuh-user]#
```

Figure 55

Webhook Shuffle intégré dans ossec.conf.

```
TELEGRAM_CHAT_ID = "1968246758"

def send_telegram(message):
    url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
    data = {"chat_id": TELEGRAM_CHAT_ID, "text": message, "parse_mode": "Markdown"}
    try:
        r = requests.post(url, json=data, timeout=15)
        return r.status_code == 200
    except Exception as e:
        sys.stderr.write(f"Telegram error: {str(e)}\n")
        return False

def format_alert(alert):
    rule = alert.get('rule', {})
    agent = alert.get('agent', {})
    level = rule.get('level', 0)
    desc = rule.get('description', 'N/A')
    agent_name = agent.get('name', 'N/A')
    agent_ip = agent.get('ip', 'N/A')
    timestamp = alert.get('timestamp', 'N/A')

    if level >= 12:
        severity = "CRITIQUE"
    elif level >= 7:
        severity = "ELEVE"
    elif level >= 4:
        severity = "MOYEN"
    else:
        severity = "FAIBLE"

    msg = f"*[SOC PANESS] Alerte {severity}*{n}---{n}Règle: {rule.get('id','N/A')} (Niv.{level}){n}Descript
ion: {desc}{n}Agent: {agent_name} ({agent_ip}){n}Heure: {timestamp}"
    return msg[:1500]

def main():
    try:
        with open(sys.argv[1]) as f:
            alert = json.load(f)
            message = format_alert(alert)
            if send_telegram(message):
                sys.stdout.write("Alerte Telegram envoyée\n")
            else:
                sys.stdout.write("Echec envoi Telegram\n")
    except Exception as e:
        sys.stderr.write(f"Erreur: {str(e)}\n")
        sys.exit(1)

if __name__ == "__main__":
    main()
[root@wazuh-server wazuh-user]#
```

Figure 56

Script custom-whatsapp.py créé sur wazuh-server.

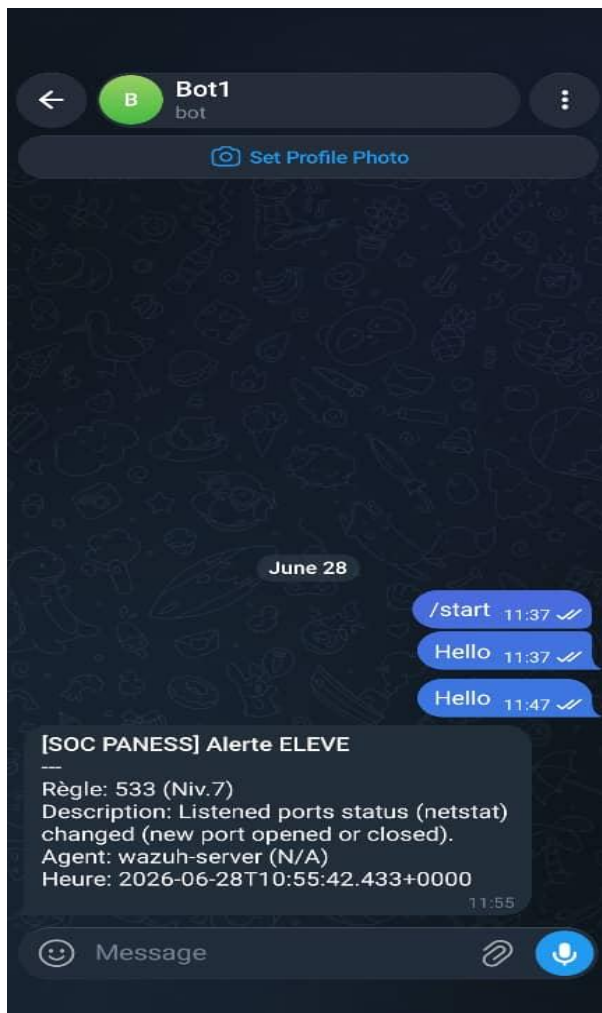


Figure 57
Message du bot Telegram reçu suite à alerte critique de Wazuh.

```

2026/06/29 08:40:15 [DEBUG] Running on shuffle-backend:5001
2026/06/29 08:40:44 [INFO] Configuring Opensearch indexes for scaling
2026/06/29 08:40:45 [WARNING] Failed ensuring ISM rollover policy 'shuffle-rollover': status: 409, body:
{"error":{"root_cause":[{"type":"version_conflict_engine_exception","reason":"[shuffle-rollover]: versi
on conflict, document already exists (current version [1])","index":"opendistro-ism-config","shard":"0"
,"index_uuid":"VeEOsL-9RcqcVLTqhWfAGw"}],"type":"version_conflict_engine_exception","reason":"[shuffl
e-rollover]: version conflict, document already exists (current version [1])","index":"opendistro-ism-confi
g","shard":"0","index_uuid":"VeEOsL-9RcqcVLTqhWfAGw"},"status":409}
2026/06/29 08:40:45 [WARNING] Failed ensuring ISM rollover policy 'shuffle-rollover' in prefix fix: stat
us: 409, body: {"error":{"root_cause":[{"type":"version_conflict_engine_exception","reason":"[shuffl
e-rollover]: version conflict, document already exists (current version [1])","index":"opendistro-ism-confi
g","shard":"0","index_uuid":"VeEOsL-9RcqcVLTqhWfAGw"}],"type":"version_conflict_engine_exception","reaso
n":"[shuffle-rollover]: version conflict, document already exists (current version [1])","index":"opend
istro-ism-config","shard":"0","index_uuid":"VeEOsL-9RcqcVLTqhWfAGw"},"status":409}
2026/06/29 08:40:45 [INFO] Prefix repair before init: expected aliases=12 found=12
2026/06/29 08:40:45 [INFO] Rollover executed successfully for workflowexecution
2026/06/29 08:40:45 [INFO] Rollover executed successfully for datastore_ngram
2026/06/29 08:40:45 [INFO] Rollover executed successfully for org_cache
2026/06/29 08:40:45 [INFO] Rollover executed successfully for org_cache_revisions
2026/06/29 08:40:45 [INFO] Rollover executed successfully for notifications
2026/06/29 08:40:45 [INFO] Rollover executed successfully for shuffle_logs
2026/06/29 08:40:45 [INFO] Rollover executed successfully for environments
2026/06/29 08:40:45 [INFO] Rollover executed successfully for org_statistics
2026/06/29 08:40:45 [INFO] Rollover executed successfully for workflowapp
2026/06/29 08:40:45 [INFO] Rollover executed successfully for workflow
2026/06/29 08:40:45 [INFO] Rollover executed successfully for workflow_revisions
2026/06/29 08:40:45 [INFO] Rollover executed successfully for datastore_category
2026/06/29 08:40:45 [WARNING] Failed ensuring ISM rollover policy 'shuffle-rollover' in prefix fix: stat
us: 409, body: {"error":{"root_cause":[{"type":"version_conflict_engine_exception","reason":"[shuffl
e-rollover]: version conflict, document already exists (current version [1])","index":"opendistro-ism-confi
g","shard":"0","index_uuid":"VeEOsL-9RcqcVLTqhWfAGw"}],"type":"version_conflict_engine_exception","reaso
n":"[shuffle-rollover]: version conflict, document already exists (current version [1])","index":"opend
istro-ism-config","shard":"0","index_uuid":"VeEOsL-9RcqcVLTqhWfAGw"},"status":409}
2026/06/29 08:40:45 [INFO] Alias verification after init passed: expected aliases=12 found=12
2026/06/29 08:40:46 [INFO] Setting up 0 schedule(s)
2026/06/29 08:40:46 [INFO] Starting cloud schedules for orgs if enabled!
2026/06/29 08:40:46 [INFO] Skipping org syncCheck for 'd2aa9232-e0fb-44e4-b05f-8c97262e93bf' because syn
c isn't set (1)
2026/06/29 08:40:46 [WARNING] No apikey found for cleanup. Skipping cleanup schedule.
2026/06/29 08:40:46 [INFO] Getting and validating workflowapps. Got 0 with err <nil>
2026/06/29 08:40:46 [DEBUG] Downloading default apps as 0 were found
2026/06/29 08:40:46 [INFO] Skipping download of apps since no URL is set. Default would be https://github
.com/shuffle/python-apps
2026/06/29 08:40:46 [DEBUG] Getting apps from url 'https://github.com/shuffle/python-apps'
2026/06/29 08:42:47 [WARNING] Skipping build of Yara to later
2026/06/29 08:42:47 [INFO] Starting build of 28 containers (FIRST)
2026/06/29 08:42:48 [INFO] Setting up memory build structure for folder: email/1.3.0/
2026/06/29 08:42:48 [INFO] Building email/1.3.0/ with proxy "... Tags: "frikky/shuffle:email_1.3.0". This
may take up to a few minutes.
root@soc-services:/opt/Shuffle#

```

Figure 58

Notification Shuffle pour vérification VirusTotal sur Telegram.

Chapitre 5 : Validation et Résultats

5.1 Scénarios de Test

Afin de valider l'architecture déployée, quatre scénarios d'attaque contrôlés ont été exécutés dans l'environnement de laboratoire. Chaque scénario vise à vérifier une chaîne de détection et de réponse spécifique.

5.1.1 Scénario 1 : Détection et Réponse à un Malware

Un fichier de test (EICAR) a été déposé dans un répertoire surveillé sur agent-linux. Le module FIM a détecté la création, déclenchant YARA et VirusTotal. La détection YARA a entraîné la suppression automatique, tandis que l'alerte enrichie par DeepSeek a été transmise à IRIS et notifiée sur Telegram.

```
[root@wazuh-server wazuh-user]# grep -E 'id":"(100301|108001|108002)' /var/ossec/logs/alerts/alerts.js
on | grep "eicar_final6" | head -3
{"timestamp":"2026-06-29T05:26:43.838+0000","rule":{"level":5,"description":"FIM: Fichier ajouté dans /home/raoulbiga/eicar_final6.txt","id":"100301","firedtimes":2,"mail":false,"groups":["local"]},"agent":{"id":"001","name":"agent-linux","ip":"192.168.100.30"},"manager":{"name":"wazuh-server"},"id":"1782710803.453028","full_log":"File '/home/raoulbiga/eicar_final6.txt' added\nMode: realtime\n","syscheck":{"path":"/home/raoulbiga/eicar_final6.txt","mode":"realtime","size_after":"64","perm_after":"rw-r--r--","uid_after":"0","gid_after":"0","md5_after":"a1310ef50438e16b804dcd2013598853","sha1_after":"130c7c8332e79ac91b38f67d5ac953a065a12fdd","sha256_after":"0c61f17897eae193adb583f8059aae021691114e713e42d44599e65be3d4c183","uname_after":"root","gname_after":"root","mtime_after":"2026-06-28T11:23:10","inode_after":"654104","event":"added"},"decoder":{"name":"syscheck_new_entry"},"location":"syscheck"}
{"timestamp":"2026-06-29T05:26:45.883+0000","rule":{"level":5,"description":"YARA: Fichier malveillant supprimé: /home/raoulbiga/eicar_final6.txt","id":"108002","firedtimes":2,"mail":false,"groups":["local"]},"agent":{"id":"001","name":"agent-linux","ip":"192.168.100.30"},"manager":{"name":"wazuh-server"},"id":"1782710805.454234","full_log":"wazuh-YARA: INFO - Successfully deleted /home/raoulbiga/eicar_final6.txt","decoder":{"name":"YARA_decoder"},"data":{"YARA":{"log_type":"INFO","file_deleted":"/home/raoulbiga/eicar_final6.txt"},"location":"/var/ossec/logs/active-responses.log"}
{"timestamp":"2026-06-29T05:26:45.883+0000","rule":{"level":10,"description":"YARA: Fichier \"\" correspondant à eicar_test","id":"108001","firedtimes":2,"mail":false,"groups":["local"]},"agent":{"id":"001","name":"agent-linux","ip":"192.168.100.30"},"manager":{"name":"wazuh-server"},"id":"1782710805.454592","full_log":"wazuh-YARA: INFO - Scan result: eicar_test [] /home/raoulbiga/eicar_final6.txt","decoder":{"name":"YARA_decoder"},"data":{"YARA":{"log_type":"INFO","rule_name":"eicar_test"},"location":"/var/ossec/logs/active-responses.log"}
[root@wazuh-server wazuh-user]#
```

Figure 59

Chronologie des alertes Wazuh : FIM → YARA → suppression automatique.

5.1.2 Scénario 2 : Balayage Réseau

Un scan nmap -A a été lancé depuis kali-attacker vers agent-linux. Suricata a identifié les signatures caractéristiques, l'alerte a été transmise à Wazuh, classée comme reconnaissance réseau, et acheminée vers IRIS.

```
[root@wazuh-server wazuh-user]# sudo tail -100 /var/ossec/logs/alerts/alerts.json | grep '"id":"86601"' | tail -1 | jq -r '"\(.timestamp) | \(.rule.description) | \(.data.src_ip) -> \(.data.dest_ip) | \(.data.alert.signature)'"
2026-06-29T14:42:53.054+0000 | Suricata: Alert - ET SCAN Potential SSH Scan OUTBOUND | 192.168.100.40 -> 192.168.100.10 | ET SCAN Potential SSH Scan OUTBOUND
[root@wazuh-server wazuh-user]#
```

Figure 60

Alerte Suricata/Wazuh générée suite au scan nmap.

5.1.3 Scénario 3 : Commande Suspecte sur Linux

La commande nc (netcat) a été exécutée sur agent-linux. Auditd l'a journalisée, Wazuh l'a comparée à la liste CDB et a généré une alerte niveau 12, transmise à IRIS et notifiée sur Telegram.

```
[root@wazuh-server wazuh-user]# sudo grep '"id":"100210"' /var/ossec/logs/alerts/alerts.json | grep "agent-linux" | tail -1 | jq -r '"\(.timestamp) | Niveau: \(.rule.level) | Règle: \(.rule.id) | \(.rule.description) | Agent: \(.agent.name)'"
2026-06-29T11:31:00.852+0000 | Niveau: 12 | Règle: 100210 | Auditd: suspicious command execution detected (nc) | Agent: agent-linux
[root@wazuh-server wazuh-user]#
```

Figure 61

Alerte niveau 12 confirmant la détection de « nc » via la liste CDB.

5.1.4 Scénario 4 : Force Brute SSH

Plusieurs tentatives SSH avec des identifiants invalides ont été lancées depuis kali-attacker. Wazuh a détecté la répétition d'échecs, vérifié l'IP source auprès de MISP et transmis l'alerte à IRIS.

```
[root@wazuh-server wazuh-user]# sudo grep '"id":"5710"' /var/ossec/logs/alerts/alerts.json | grep "192.168.100.40" | tail -1 | jq -r '"\(.timestamp) | Niveau: \(.rule.level) | Règle: \(.rule.id) | \(.rule.description) | IP source: \(.data.srcip) | Agent: \(.agent.name)'"
2026-06-29T15:01:31.849+0000 | Niveau: 5 | Règle: 5710 | sshd: Attempt to login using a non-existent user | IP source: 192.168.100.40 | Agent: agent-linux
[root@wazuh-server wazuh-user]#
```

Figure 62

Détection de force brute SSH avec corrélation MISP de l'IP source.

5.2 Synthèse des Résultats

Les quatre scénarios ont confirmé que chaque composant du SOC fonctionne de manière autonome et que les intégrations entre outils opèrent comme prévu. La plateforme a démontré sa capacité à détecter des menaces variées (malware, reconnaissance réseau, commandes suspectes, force brute) et à déclencher des réponses adaptées.

```
[root@wazuh-server wazuh-user]# sudo tail -100 /var/ossec/logs/alerts/alerts.json | jq -r 'select(.rule != null)
.agent.name)' | tail -10
2026-06-29T15:15:09.931+0000 | Règle: 5501 | Niveau: 3 | PAM: Login session opened. | Agent: wazuh-server
2026-06-29T15:15:17.956+0000 | Règle: 5502 | Niveau: 3 | PAM: Login session closed. | Agent: wazuh-server
2026-06-29T15:15:29.964+0000 | Règle: 5402 | Niveau: 3 | Successful sudo to ROOT executed. | Agent: wazuh-server
2026-06-29T15:15:29.964+0000 | Règle: 5501 | Niveau: 3 | PAM: Login session opened. | Agent: wazuh-server
2026-06-29T15:15:37.973+0000 | Règle: 5502 | Niveau: 3 | PAM: Login session closed. | Agent: wazuh-server
2026-06-29T15:19:04.327+0000 | Règle: 5402 | Niveau: 3 | Successful sudo to ROOT executed. | Agent: wazuh-server
2026-06-29T15:19:04.328+0000 | Règle: 5501 | Niveau: 3 | PAM: Login session opened. | Agent: wazuh-server
2026-06-29T15:19:04.328+0000 | Règle: 5502 | Niveau: 3 | PAM: Login session closed. | Agent: wazuh-server
2026-06-29T15:20:22.589+0000 | Règle: 5402 | Niveau: 3 | Successful sudo to ROOT executed. | Agent: agent-linux
2026-06-29T15:20:22.618+0000 | Règle: 5501 | Niveau: 3 | PAM: Login session opened. | Agent: agent-linux
[root@wazuh-server wazuh-user]#
```

Figure 63

Vue d'ensemble des alertes dans le Dashboard Wazuh lors des tests.



Figure 64
Notifications Telegram reçues correspondant aux quatre scénarios.

5.3 Limites Observées

Plusieurs limites ont été constatées. La maintenance d'une pile multi-outils open-source exige une expertise technique significative. Les règles de détection requièrent un ajustement continu pour réduire les faux positifs. Les performances restent dépendantes des ressources allouées à l'environnement virtualisé, ce qui devra être dimensionné correctement lors d'un passage en production.

Conclusion générale et perspectives

Bilan

Ce stage au sein de PANESS a permis la conception, le déploiement et la validation d'une architecture SOC complète basée exclusivement sur des outils open-source. En plaçant Wazuh au cœur du dispositif, l'écosystème construit intègre la détection réseau (Suricata), la supervision des terminaux (Auditd, Sysmon, FIM), l'analyse de malware enrichie par IA (YARA, VirusTotal, DeepSeek), le renseignement sur les menaces (MISP), la gestion des incidents (DFIR-IRIS) et l'automatisation avec notification en temps réel (Shuffle, Telegram). Les objectifs spécifiques fixés ont tous été atteints et validés par des scénarios de test représentatifs.

Contributions

La contribution principale de ce projet est la démonstration concrète qu'il est possible de construire un SOC puissant et fonctionnel avec un budget logiciel nul, fournissant un modèle viable pour les organisations africaines à ressources limitées que PANESS accompagne.

Perspectives d'Amélioration

- Développer des playbooks Shuffle plus complexes couvrant des scénarios multi-étapes.
- Étendre la supervision aux environnements cloud (AWS, Azure) et conteneurisés (Docker, Kubernetes).
- Intégrer des sources de Threat Intelligence supplémentaires dans MISP (flux OSINT, ISAC).
- Mettre en place des mécanismes de corrélation avancés basés sur l'apprentissage automatique.
- Préparer la migration de cette architecture de laboratoire vers un environnement de production au sein de PANESS.

RÉFÉRENCES BIBLIOGRAPHIQUES

- Bhatt, S., Manadhata, P. K., & Zomlot, L. (2014). The operational role of security information and event management systems. *IEEE Security & Privacy*, 12(5), 35–41. <https://doi.org/10.1109/MSP.2014.103>
- DFIR-IRIS. (2024). *IRIS — Digital forensics incident response platform*. <https://github.com/dfir-iris/iris-web>
- Hutchins, E. M., Cloppert, M. J., & Amin, R. M. (2011). Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. *Leading Issues in Information Warfare & Security Research*, 1(1), 80–106.
- IBM Security. (2024). *X-Force Threat Intelligence Index 2024*. IBM Corporation. <https://www.ibm.com/reports/threat-intelligence>
- MISP Project. (2024). *MISP — Malware Information Sharing Platform & Threat Sharing*. <https://www.misp-project.org>
- Shuffle. (2024). *Shuffle — Open source SOAR platform*. <https://shuffler.io>
- Suricata. (2024). Suricata open source IDS/IPS/NSM engine. Open Information Security Foundation. <https://suricata.io>
- VirusTotal. (2024). *VirusTotal — Free online virus, malware and URL scanner*. <https://www.virustotal.com>
- Wagner, C., Dulaunoy, A., Wagener, G., & Iklody, A. (2016). MISP: The design and implementation of a collaborative threat intelligence sharing platform. *Proceedings of the 2016 ACM on Workshop on Information Sharing and Collaborative Security* (pp. 49–56). ACM. <https://doi.org/10.1145/2994539.2994542>
- Wazuh. (2024). *Wazuh — The open source security platform*. <https://wazuh.com>

TABLE DES MATIÈRES

Dédicace	1
Remerciements	3
Résumé	4
Abstract	5
Sommaire	6
Liste des figures	7
Liste des tableaux	12
Liste des annexes	13
Liste des abréviations	14
Introduction générale	15
Chapitre 1 — Présentation de l'entreprise d'accueil : PANESS	21
1.1. Historique et présentation générale	16
1.2. Les filiales du groupe	16
1.3. Cadre du stage	22
Chapitre 2 — Cadre conceptuel	18
2.1. Définition des concepts clés	18
2.2. Nécessité d'un SOC dans le contexte actuel	19
Chapitre 3 — Problématique, objectifs et architecture retenue	20
3.1. Problématique	20
3.2. Objectifs du projet	20
3.3. Choix des Outils et Justification	21
3.4. Architecture Globale Retenue	22
Chapitre 4 — Mise en œuvre de l'infrastructure SOC	23
4.1. Phase 1 — Infrastructure de virtualisation et réseau	23
4.2. Phase 2 — Déploiement de Wazuh	24
4.3. Phase 3 — Agents Wazuh	25
4.4. Phase 4 — Surveillance des terminaux (FIM, Auditd, Sysmon)	26
4.5. Phase 5 — Détection réseau avec Suricata	27
4.6. Phase 6 — Analyse malware : YARA, VirusTotal et IA DeepSeek	28
4.7. Phase 7 — Threat Intelligence avec MISP	29
4.8. Phase 8 — Gestion des incidents avec DFIR-IRIS	30
4.9. Phase 9 — Automatisation et notification Telegram	31
Chapitre 5 — Validation et résultats	56
5.1. Scénarios de test	56
5.2. Synthèse des résultats	58
5.3. Limites observées	58
Conclusion générale et perspectives	60

Références bibliographiques	61
Annexes	32
Table des matières	62